

Software Engineering by and for Humans in an AI Era

SILVIA ABRAHÃO, Department of Computer Science, Universitat Politècnica de València, Valencia, Spain

JOHN GRUNDY, Software Systems and Cybersecurity, Monash University, Melbourne, Australia

MAURO PEZZÈ, Faculty of Informatics, Università della Svizzera Italiana, Lugano, Switzerland and

CONSTRUCTOR Institute of Technology, Schaffhausen, Switzerland and Dipartimento di Informatica

Sistemistica e Comunicazione, Università degli Studi di Milano-Bicocca, Milano, Italy

MARGARET-ANNE STOREY, Computer Science, University of Victoria, Victoria,

British Columbia, Canada

DAMIAN A. TAMBURRI, Technische Universiteit Eindhoven, Eindhoven, Netherlands

The landscape of software engineering is undergoing a transformative shift driven by advancements in machine learning, Artificial Intelligence (AI), and autonomous systems. This roadmap article explores how these technologies are reshaping the field, positioning humans not only as end users but also as critical components within expansive software ecosystems. We examine the challenges and opportunities arising from this human-centered paradigm, including ethical considerations, fairness, and the intricate interplay between technical and human factors. By recognizing humans at the heart of the software lifecycle—spanning professional engineers, end users, and end user developers—we emphasize the importance of inclusivity, human-aligned workflows, and the seamless integration of AI-augmented socio-technical systems. As software systems evolve to become more intelligent and human-centric, software engineering practices must adapt to this new reality. This article provides a comprehensive examination of this transformation, outlining current trends, key challenges, and opportunities that define the emerging research and practice landscape, and envisioning a future where software engineering and AI work synergistically to place humans at the core of the ecosystem.

CCS Concepts: • **Software and its engineering** → **Software development techniques**; **Designing software**; • **Human-centered computing** → **Interaction paradigms**; **Collaborative and social computing systems and tools**; • **Computing methodologies** → **Artificial intelligence**;

Additional Key Words and Phrases: Software engineering, Human and social aspects, Large language models

John Grundy is supported by ARC Laureate Fellowship FL190100035. Silvia Abrahão is supported by State Research Agency under grant PID2022 140106NB I00 (UCI Adapt) and Generalitat Valenciana under grant CIAICO/2021/303 (AKILA). Mauro Pezzè is supported by the Swiss National Foundation under grant SNSF 200021_215487

Authors' Contact Information: Silvia Abrahão, Department of Computer Science, Universitat Politècnica de València, Valencia, Spain; e-mail: sabrahao@dsic.upv.es; John Grundy, Software Systems and Cybersecurity, Monash University, Melbourne, Australia; e-mail: john.grundy@monash.edu; Mauro Pezzè (corresponding author), Faculty of Informatics, Università della Svizzera Italiana, Lugano, Switzerland and CONSTRUCTOR Institute of Technology, Schaffhausen, Switzerland and Dipartimento di Informatica Sistemistica e Comunicazione, Università degli Studi di Milano-Bicocca, Milano, Italy; e-mail: mauro.pezze@usi.ch; Margaret-Anne Storey, Computer Science, University of Victoria, Victoria, British Columbia, Canada; e-mail: mstorey@uvic.ca; Damian A. Tamburri, Technische Universiteit Eindhoven, Eindhoven, Netherlands; e-mail: d.a.tamburri@tue.nl.



This work is licensed under Creative Commons Attribution International 4.0.

© 2025 Copyright held by the owner/author(s).

ACM 1557-7392/2025/5-ART129

<https://doi.org/10.1145/3715111>

ACM Reference format:

Silvia Abrahão, John Grundy, Mauro Pezzè, Margaret-Anne Storey, and Damian A. Tamburri. 2025. Software Engineering by and for Humans in an AI Era. *ACM Trans. Softw. Eng. Methodol.* 34, 5, Article 129 (May 2025), 46 pages.

<https://doi.org/10.1145/3715111>

1 Introduction

The last decade has seen a sudden acceleration of many new technologies that radically change software production and its impact on human life, work, and society. Generative AI, **Extended Reality (XR)**, and the evolving Internet of Things have opened major new frontiers. These have upset current and future research and practice with a disruptive revolution greatly beyond the significant changes of even the last decade. Generative AI has already redefined the production of software systems with improvement of productivity and sometimes disruptive side effects [75, 84]. Current tools and practices are just scratching the surface of a deep revolution that may have major impacts on the way we engineer software systems in the next decades [45, 57, 127]. Generative AI, XR, and the Internet of Things enable new ways for humans to interact with their diverse software systems. **Machine Learning (ML)**-powered systems and bots autonomously evolve and interact with humans in many ways that were unforeseen till few years ago. Increasingly, humans are beginning to interact more seamlessly and unconsciously with a range of complex software systems simply by living in increasingly software-driven smart cities. XR and the Internet of Things enable human–system interactions far beyond the classic keyboard–mouse–display interactions.

Generative AI empowers new scenarios for both designing software systems and living within human-centric systems. However, these developments are not without a variety of risks, including concerns over what **Software Engineering (SE)** jobs will look like, AI bias, ethical and responsible engineering practices, inclusive solutions, and how to best educate future AI-powered system engineers [85, 91, 129, 146].

In the *Artificial Intelligence for Software Engineering: The Journey so far and the Road Ahead* paper in this issue, Chatzigeorgiou et al. [28] discuss the impact of AI on SE practices. In this article, we focus on human factors in the *area of AI* and propose a roadmap for research on human aspects in both SE development processes and human-centric software systems. We discuss the way new AI-powered tools dramatically change the development of software systems and the dynamics of development teams, the SE profession, and SE education. We discuss the need for software adaptation for diverse end users and usage contexts by leveraging AI and human interaction, and the validation of human-centric self-adaptive ecosystems. We argue how human-centric, self-adaptive ecosystems reshape the role of software in human life and society.

We illustrate the impact of generative AI and human-centric self-adaptive ecosystems throughout Sections 2 and 3, with McLuhan’s Tetrad [109]. The *Tetrad* is a diagram that the Canadian philosopher Herbert Marshall McLuhan defined in the mid-70s to explain the disruptive effect of media on the society. McLuhan observed that disruptive technologies always follow intrinsic laws: they amplify certain human abilities, render previous technologies obsolete, and when pushed to their extremes, create a need for new innovations to address emergent new challenges. In this article, we use McLuhan’s Tetrad to highlight some of the key effects of disruptive innovation, as Storey et al. suggest in their *Disruptive Research Playbook* [151]. Figure 1 shows the four diamonds that illustrate the effect of the technology (in the center in the diagram) by indicating how the new technology *enhances* software development and systems, *obsolesces* technologies and approaches, *reverses* common practice, and *retrieves* past results, to propose a roadmap for SE by and for humans systems in the AI era. For example, the Internet, in the middle of Figure 1,

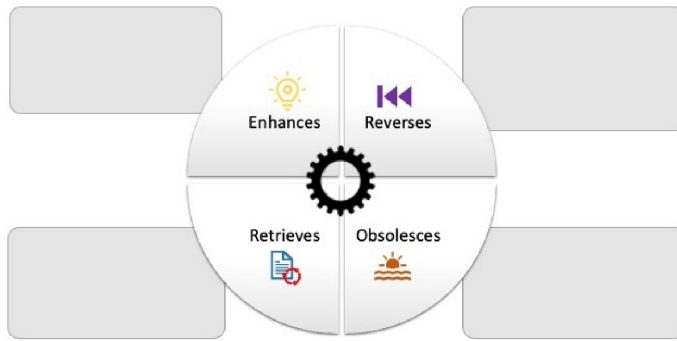


Fig. 1. McLuhan's Tetrad.

has amplified distributed development (the enhances diamond) and has driven out of practice tarballs and zip files (the obsolesces diamond), by offering a new support to collaborative development, it has recovered standardized processes to organize distributed development (the retrieves diamond) and it has emphasized social coding when pushed to the limit (the reverses diamond).

2 Software Development in the Era of AI

Software development is one of the most creative, complex but also rewarding forms of knowledge work. Facilitated by the Internet, tens of thousands of developers are able to work together, directly and indirectly, to produce long-lived multi-version complex systems (such as Windows and Google) with millions of static and dynamic system and software dependencies that change rapidly over time to keep up with the demand for new or different user features.

To keep pace with such complexity, developers innovate and adopt new processes, novel technologies, and powerful automation to augment their capabilities for creating and orchestrating how software system versions are managed by these distributed networks of thousands of developers. These innovations, such as the Internet and social coding tools, and processes, such as agile, change not just the speed and quality of the software being developed, but they also alter the very essence of how software is authored by developers, and the ways they work together.

Generative AI-based tools are the latest disruption to reshape how software is being engineered. We use McLuhan's Tetrad, introduced in Section 1, as a lens for laying out a multi-path roadmap of how AI will disrupt development practices and experiences in expected and unexpected ways, as well as potentially both positive and negative ways (Figure 2). We may anticipate that AI may *enhance* the speed and creativity of developers, and remove (makes *obsolete*) friction and tedium in work and the need to know lower level technical details. However, it may also *retrieve* artifacts and activities from times before, for instance, interaction with ICQ style chatbots and more reliance on “natural language” as the design language. What it will *reverse* into is also of concern—loss of control, trust, and quality over time.

In the following we first lay out how AI is reshaping how we conceptualize developer productivity and experience (Section 2.1), next we explore how AI is disrupting the tools used in SE (Section 2.2), how AI is transforming how humans and machines together create software collaboratively (Section 2.3), how AI will impact *who* will be designing software and the SE profession (Section 2.4), and finally how AI will impact SE education (Section 2.5).

2.1 The Impact of AI on the Developer Productivity and Experience

Improving tools and processes to enhance developer productivity and their experience has been an ongoing quest since even before the term “software engineering” was coined in the NATO 1968 conference on the software crisis [119]. Before we discuss how AI is impacting developer tooling

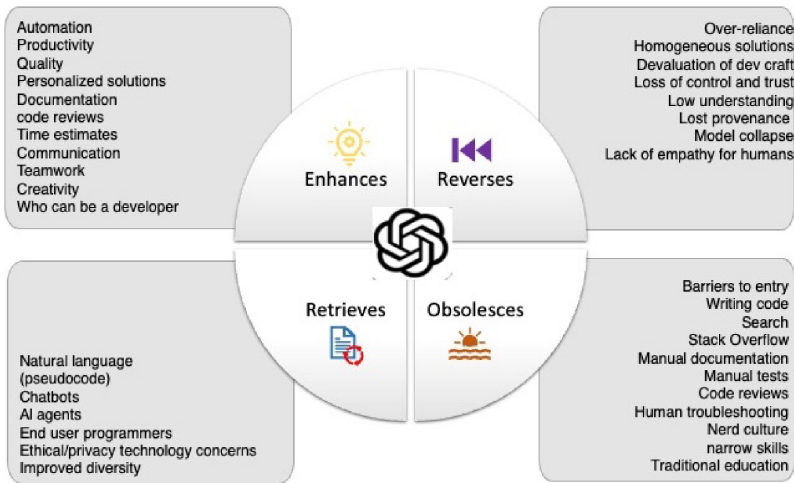


Fig. 2. An idealized view of the disruptive effects of AI.

(see Section 2.2), we revisit some historical landmarks in developer productivity and experience, the ongoing challenges we face today, and a roadmap for future work to be considered about how AI impacts developer productivity and experience.

2.1.1 Trends and Historical Landmarks. Frederick J. Brooks was one of the first authors to write about developer productivity in his famous 1975 *Mythical Man-Month* book [23].¹ The essays in his book, based on his experience managing a large, complex project at IBM on the OS/360, led to several timeless project management insights that still hold today. He recognized that the inherent essential complexity in software cannot be tamed by simply adding more developers to the project, in part due to communication and coordination overhead and difficulties maintaining an understanding of the overall design of a complex project. Brooks also famously noted that there is likely no *silver bullet* to significantly improve developer productivity. He argues that tools, languages, and processes can only help us address what he termed accidental complexity in software, but do little to address the essential complexity inherent in the problem and solution domains. Brooks also discussed how software is not visible until there is an early prototype, and it is only with these early prototypes, that users will discover what they want, putting pressure on the requirements and the software to change. Brooks recognized many of the challenges we still experience with versioning, documentation, project estimation, administrative overhead, and increasing errors over time.

Gerald M. Weinberg, another luminary, also discussed some of the same important themes about human factors in his landmark 1971 *The Psychology of Programming* book [169] and noted that software development is a human activity, rather than a technical activity. He talks about the psychological challenges in understanding program complexity, and the importance of effective communication and positive team dynamics for distinguishing high-performing software teams. He also mentioned the importance of continuous improvement, made possible by learning during debugging. He further proposed innovative ideas for developer documentation, whereby developers would write customized answers to specific developer questions that capture the context and needs of the questioning developers. This idea is prescient of Stack Overflow, a tool that did not appear for almost 40 years. The concepts behind continuous SE and collaborative and interactive learning are still the cornerstones of successful development today.

¹This is the reference to the first of several editions.

The argument that software development is much more than the technical details captured by code was also strongly advocated by Peter Naur in his famous 1986 essay [118]. Naur considered software as *theories in the minds of developers*: theories about the problem domain, theories about how the software implements solutions to those problems, and theories about how the program can be evolved to support new and changing features. He also recognized the importance of mentorship and knowledge sharing among team members.

Another seminal book on software developer productivity was DeMarco and Lister's 1987 *Peopleware: Productive Projects and Teams* book [36]. DeMarco and Lister discussed the importance of developer psychology, team work, and the developer's environment. They also recognized the importance of project management and the critical role of the manager, and how intrinsic motivation, driven by a developer's personal interest and satisfaction in their work, is an important key to a team's success (even more so than bonuses and extrinsic rewards).

During the 1990s and into the 2000s and beyond, practitioners and researchers, building on the recognition of the importance of human factors and teamwork, synthesized and shared wisdom to improve programming techniques [78], more advanced programming languages [86], and programming tools (see Section 2.2). Extreme programming [17] and the agile movement [147] further led to insights of how to improve developer productivity and their experience, with practices such as refactoring, pair programming, customer focus, communication, test driven development, testing, scrum, and continuous integration showing widespread adoption with positive impacts. While at the same time, others paid significant attention to how to measure, and hopefully, better control or influence productivity [15, 82, 154].

In the past decade, there has been additional substantive research on developer productivity [163], developer happiness [69] and satisfaction [153], developer motivation [18] and developer experience [71], and widespread adoption in industry of approaches for framing and measuring productivity [50, 51].

2.1.2 Open Challenges. AI dramatically impacts on developer productivity, experience, flow, and creativity.

Productivity. How to define, understand, and measure developer productivity is an ongoing challenge, tackled by both industry and research. Not that long ago, many large companies used lines of code as a way of measuring development productivity. We have come a long way from then and realized that productivity is a complex concept and if it is to be measured (some argue it should not be!), it should be measured across many dimensions. Forsgren et al. [51] proposed that productivity should be conceptualized across at least five dimensions: developer satisfaction and well-being, performance and quality, activity (such as lines of code, code reviews done), collaboration and communication, and finally developer/project efficiency and flow. The SPACE framework emerged from decades of research on developer productivity, and it has been adopted across the industry to understand different dimensions behind improvement interventions as well as ways of measurement. SPACE has also been used as a framework for understanding the different impacts as AI (implemented with Copilot) that was adopted by GitHub developers [84]. But much remains to understand and improve how AI impacts developer productivity and developer experience. The dimensions in SPACE are one way to capture those insights, but as AI is adopted, we will expect to see new ways to measure these dimensions of productivity.

Trust and Autonomy. Important aspects of developer experience will relate to their trust in generative AI (and how it will impact the quality of their work) as well as their autonomy over how any automated developed work is done. Trust will also be influenced by how developers feel secure in how they use AI to generate code and other artifacts, as well as how using AI will influence their sense of identity [76].

Creativity. Software development is a highly creative endeavor. Researchers in our community are just starting to question how AI will enhance (or impede) creativity in the software activities of brainstorming, design, development, and analysis [80]. When developers work with AI, will they become less or more creative? How can AI support creativity at the individual and team levels, while also helping developers, stay on track with their assigned work. Jackson et al. speculate that individual creativity can be enhanced by bringing in inspirations from other disciplines, bringing perspectives from the future users of the software, and relating those ideas to architecture design issues [80]. These authors also suggest that generative AI may level the playing field and make individual personality traits less relevant, but at the same time may reduce reliance on human mentorship. How AI may further engage discussions between the end users and developers is discussed further below in this article when we consider how generative AI will impact collaboration (see Section 2.3) and is an important open challenge.

2.1.3 Roadmap. There are many open research issues about developers' productivity and experience, and the industry is hungry to know how to conceptualize these concepts and how to measure the impact of tools, processes, and practices. Specifically, the challenges mentioned above lead to the following research activities.

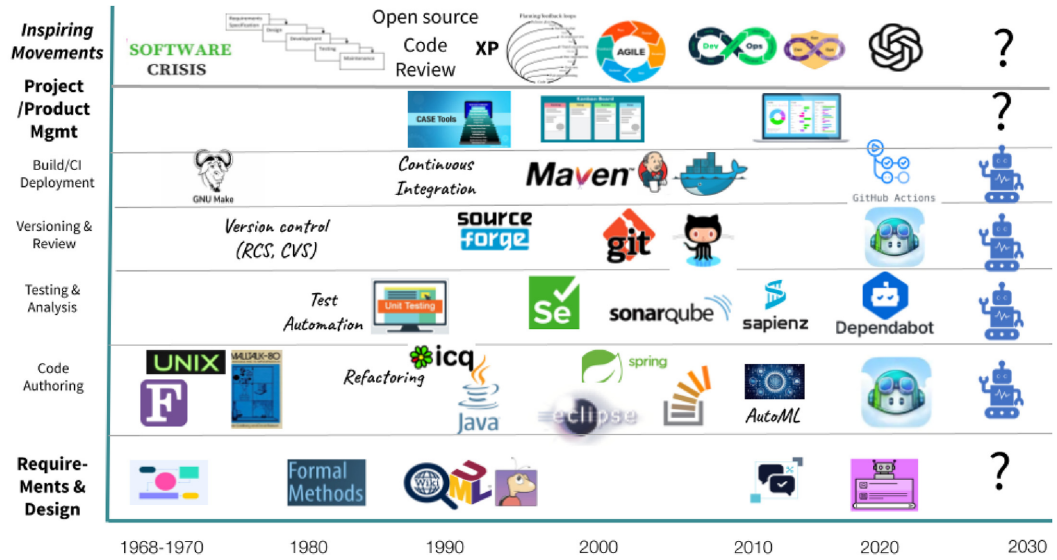
- *Which theories should we borrow and extend from other disciplines, and what new theories do we need to develop?* Both developer productivity and developer experience rely on concepts from many other disciplines, including the management sciences, **Human-Computer Interaction (HCI)**, behavioral psychology, and the social sciences. At the same time, we will have to build brand new theories and revisit older theories as AI disrupts what we know and assume about developer productivity and their experience. For example, how AI will impact developer creativity is a topic of interest.² We will need to consult and be aware of theories of creativity from other disciplines as suggested by Jackson et al. [80].
- *How to measure developer productivity and experience?* As AI is rolled out, and as it extends its reach and capabilities, we need to know how to measure its impact on developer productivity, across many different dimensions such as those captured by the SPACE framework and on developer experience. We likely need new metrics and new ways of measuring and presenting impacts on productivity, experience, flow, creativity, trust, and autonomy on developers, managers, and researchers.
- *How will AI-powered tools and developer agents impact the creative side of software development processes and work?* Creativity is an often overlooked aspect of SE work. We need to study how AI-powered developer agents impact this, especially in terms of human collaboration around producing creative solutions to currently unknown, unsolved software solutions, domains, users, and usage contexts. Will they dampen or improve creativity, and how can we ensure the latter vs. the former.

2.2 The Impact of AI on Software Development Tools

2.2.1 Software Development Trends over Time. Software developers are always on the forefront of either developing or adopting new technical innovations to support and transform their work. Indeed, SE has been assisted by automation since the first assembly language tools were constructed.

There are many different innovations that have emerged over time that support the inner loop of how developers write and test code, as well as innovations that support the outer loop of going from idea to deployed feature. Figure 3 shows a historical timeline of emerging tools that support the different phases of the software development lifecycle (both the inner and outer loops). Tools

²This topic will be explored in an upcoming Dagstuhl seminar, see <https://www.dagstuhl.de/25412>



The needs driven emergence of innovations in software developer tools that are shaped by important movements in software engineering (such as agile, devops and recently AI), with rows below representing the outer loop (requirements/design and project management) and inner loop (from code to deployment). This figure also shows the uncertainty of how these tools will evolve by 2030. For the inner loop, we expect that automation will play a bigger role by 2030, but expect humans to continue playing a large role for outer loop but be assisted by AI and collaborate with AI agents.

Fig. 3. Historic timeline of tools supporting the SE lifecycle.

that support requirements elicitation, their capture and formalization; tools and languages that support and enhance how code is authored or auto-generated; tools that help ensure the quality of current and future work through testing and analysis; tools that allow developers to review each other's work and work together by versioning their work and systems; tools that automate the build and deployment processes; and tools that help manage the project management process.

The figure clusters tools according to their main supportive role. All tools followed an arc that was shaped by inspiring movements in how SE should be done (see the top row of Figure 3). For example, the recognition that there was a looming software crisis in the late 1960s spurred the need for tools to manage software complexity and support the then recognized inevitable need for software to continually evolve. The agile movement recognized the importance of humans and how they communicate and co-create in the process of software development led to tools that emphasized social interactions and distributed development. The DevOps movement, as another example, influenced the development of continuous integration pipelines and tooling.

Over the past several years, the latest disruptive movement of generative AI has led to an increasing number of AI-powered SE support tools. The suggestion to use AI in developer tools is also not new. Rich and Waters back in 1988 proposed the "programmer's apprentice," a vision of how tooling could use AI to help developers complete a wide range of tasks [135]. Although much of this early vision was not fully realized, of late, AI is now leaving its mark on code authoring, testing, code review, build, deployment, security, and documentation tools, among others. AI is reshaping the entire tooling environment in which software engineers practice. The right side of Figure 3 indicates that AI and the use of agents or automation are likely to transform all tools across the development lifecycle, with new tools that we cannot even conceive of yet likely emerging.

2.2.2 Open Challenges. As SE systems are augmented with AI capabilities, the new opportunities they bring also lead to several open challenges. We describe these opportunities and challenges

using the different categories of tools used across the entire lifecycle of software development (as displayed in Figure 3).

Requirements Elicitation, Management, and Documentation. The mechanisms for requirements elicitation, management, and documentation are predicted to go through several transformative and disruptive changes due to the adoption of generative AI and Natural Language Processing.³ Formerly, eliciting and documenting requirements was a labor-intensive task involving time-consuming customer inquiries, and the negotiation of competing requests and design activities. Validating and modeling requirements (to ensure completeness and consistency), and tracing of requirements to code and other artifacts were also time-consuming and error-prone tasks. Indeed, more often than not, requirements were never written down, and rarely explicitly linked to associated code artifacts.

The industry has started using generative AI to model user personas and to check for consistency and compliance among requirements, conversational bots to assist in eliciting requirements, analysis of text in feedback forums and social media to identify new and changing features, text generation to assist in documentation, links to show the traceability of requirements to code, rapid prototyping to assist in brainstorming of new requirements, and the use of generative AI to help in predicting requirements to improve non-functional requirements, such as security, privacy, and performance. These new tool features are just some of the possibilities that are just around the corner.

In the academic domain, the latest conference on requirements engineering showcases many papers on how AI can be used across the field of requirements engineering,⁴ while the literature review by Hou et al. mentions several papers that have explored how **Large Language Models (LLMs)** are used in requirements engineering to date (key topics include requirements disambiguation inherent in natural language, coherence across stakeholders, requirements classification, and traceability automation) [77].

These new capabilities and opportunities, although exciting, bring new challenges—some of which may be hard to predict. More rapid elicitation of requirements puts more pressure on the design and delivery of code to meet those needs, in turn putting pressure on quality. AI-assisted or AI-automated modeling of users and requirements, automated documentation, and validation may lead to a lack of human control and oversight or recognition of biases that may emerge from the training data used. Verification of the suggested requirements and their documentation will need to be made by the “humans” involved. Designing with diversity in mind requires special attention that may be missed when large language models are used. Furthermore, although AI could be used to help address security concerns, using AI in the requirements elicitation process may introduce new threat attack vectors that may be close to impossible for the humans involved to detect without proactive monitoring.

Coding Velocity and Ease. There are already well-established tools (for instance, Copilot from GitHub), and results from empirical studies to indicate that the use of generative AI for code generation helps improve the speed of coding [84], and there are well over 100 published academic papers on the topic of code generation alone [77].

Generative AI techniques can assist in generating snippets of code, removing the need for developers to write tedious blocks of code or the need to know how to use frameworks or APIs, and also has shown potential in generating code not just from text prompts in chat interfaces but also from sketches of UIs, or from other examples of user interactions (for instance, from user stories). Beyond code generation, many techniques also assist in program synthesis (generating code from formal requirements), code completion, code documentation, APIs, code summarizations,

³<https://medium.com/@workboxtech/ai-in-requirements-engineering-clarity-and-consistency-boost-1c7fbf2d28dd>

⁴<https://conf.researchr.org/home/RE-2024>

and search. There has also been great strides in the area of program repair (generating patches for bugs in maintenance tasks), especially when generative AI is combined with static analysis [174].

The increases in coding velocity ease for both new, evolving and legacy code [77], but may introduce challenges in terms of software quality, feature and code bloat, security risks, and unnecessary technical debt. Although generative AI may increase velocity, over time it may slow things down even more⁵ as well as have disastrous effects on the environment if care is not taken in how large language models and other expensive AI techniques are used. A bigger concern may be the eventual need for a “developer” to remain in the loop when things go wrong or when the automation or AI breaks down. In these cases, having techniques to help a human debug, understand, and repair the AI-generated or repaired code may be essential given the scale and complexity of new code that is likely to be generated. That is, tools that help address the new cognitive challenges AI SE coding tools may introduce are essential to consider up front.

Finally, the focus to date has predominantly been on development speed, but there is a need to study more on the impact of these tools on developer flow experiences and their motivation [93], as well as how these tools impact developer learning of new skills [76].

Testing and Debugging. How software is tested and analyzed is undergoing rapid changes also due to the use of generative AI. Tests and test data can be generated from natural language requirements, formal specifications, prompts, conversations with bots, code, or similar systems, and user interactions. These tests, unit, integration, and end-to-end tests can be used to safeguard against regressions as future changes are made, testing both functional and non-functional requirements such as security. Indeed, large language models are already being used for vulnerability detection [77] and compliance violations during development tasks.

Generative AI can be used to determine ideal test configurations and to take into consideration performance issues when building software, and to suggest architectural changes to the underlying system to facilitate more efficient testing and analysis over time. Generative AI shows great potential in bug localization [96].

But there are some risks and challenges to be considered across the use of AI in testing, analysis, and debugging activities. There may, over time, be too much reliance and trust on generated tests and bug detection/localizations, and the data that the generated tests and recommendations are trained from may be limited, especially in the face of a rapidly evolving software system and user base. Developer knowledge about how to test and debug may erode over time, and they may need new skills to use the AI to effectively test and debug tomorrow’s software systems. Debugging, in the face of subtle errors, may be substantially more difficult when there is a complex and not well-understood testing or verification framework being used. Safeguarding also against the “bystander” effect is something to be wary about. Developers may become complacent over time and possibly ignore signals that indicate things are not working right or that there have been security concerns introduced during the process of testing, quality analysis, and debugging. Understanding human aspects of this new generation of testing and debugging tools is critical.

Continuous Integration and Deployment. Modern software development relies on continuous integration to support hundreds (and even thousands) of developers pushing changes to multiple versions of software across one or more software repositories. Ensuring that developers can work on the system at the same time (or on parts others depend on) is made possible by build tools and merge conflict features. Tools such as GitHub further support developers collaborating and sharing insights on what they are working on and the changes that have been done, and the rationale for these changes, through work items and issues. Continuous integration is a socio-technical

⁵Just as the introduction of the car speed up transportation until too many cars were on the road.

activity [43] that requires the orchestration of both human and tools to achieve speed and quality advantages it provides.

Code review is an essential practice to ensure that developers check and validate each other's changes. Code review is not just about verifying code, but it is also about sharing awareness and knowledge of what others are doing and knowledge about some of the code as it is developed over time. No single developer will know the entire system, but having multiple developers work on and review the same parts of the code, means that knowledge is distributed in a way that supports change over time. However, code review is often a bottleneck in rapid software development, and much code review work is to check for trivial matters. Because of this, there is some ongoing research on how AI can automate or augment code review activities. However, the limitation of using LLMs to automate or augment code review is that it may reduce the other human and social benefits attained from doing manual code review—the advantages of knowledge transfer, skill acquisition, awareness of others' work, connections made between developers, and co-ownership across a code base may be reduced.

Project and Product Management. The use of generative AI for project and product management is an emerging topic of interest across the industry with limited attention by researchers so far [77]. Generative AI shows potential to support planning, cost estimations, tracking of work, communication, coordination, and decision-making [77]. Some of the challenges for these tools are similar to challenges we already mentioned above including over reliance on automation, lack of accountability, bias and reduced oversight by humans, and lack of communication and awareness. When any system is automated, end users often suffer at the expense of more efficiency for the producers or owners of the system. The reduction in human communication ability to deal with “exceptions” often leads to disastrous friction for the end users. These concerns need to be considered if and when AI is used for automating parts of the project management process.

2.2.3 Roadmap. Extensive research efforts are already well underway to study the impact of AI on the tooling landscape that supports the entire SE lifecycle. Inspired by McLuhan's laws of technology that are captured by his tetrad, we frame some future research avenues our community can continue or start to explore in terms of the different ways AI can be used to support SE activities and how we can go about understanding its impact.

- *How to leverage software knowledge to improve LLMs for SE?* Recent research on how to go beyond treating programs as text [174] and that leverage program presentation techniques, dynamic analysis, and semantic analysis show great potential for improving how LLMs will support code generation, program synthesis, program repair, program comprehension, software evolution, code review, feature testing, security testing, and build performance. That is, these techniques, perhaps put aside as AI was first introduced, should be “brought back” (to use McLuhan's term) to augment and enhance how the AI models are used.
- *How to leverage domain knowledge and the development history to improve SE tooling?* The research on **Mining Software Repositories (MSR)** has demonstrated how knowledge from software repositories (for instance, hosted on GitHub), and communication channels, such as Stack Overflow and GitHub coding, can be leveraged to ease software evolution. These knowledge resources are being used indirectly by large language models, but there is potential to directly revisit and use text mining and summarization techniques from MSR research in combination with large language models to provide further insights that can facilitate process understanding and tool enhancements.
- *How to capture and leverage developer context to enhance how AI improves Engineering tools?* For AI tools to provide the right support at the right time, they will need much more context

about the development tasks and the developers' context—what tasks did they do before, what other tasks are they doing, what do they know, who do they know, and what are their values. As researchers we need to study what context is needed, how to capture that context, and how to use that context in a way that does not impinge on a developer's privacy, while supporting them in a way that feels natural and allows them to feel in control and supported (we discuss these aspects more below). That is, capturing this context will further “enhance” how the AI will improve its use in SE tooling.

- How to leverage and synthesize AI support across the entire lifecycle of engineering tools?* The biggest change that is mostly likely to happen is the use of integrated AI-powered assistants (such as GitHub Co-pilot) that is integrated across the entire tool chain. There is some ongoing work and progress on this direction by Tufano et al. where they propose an AI-driven framework called “AutoDev” for the “autonomous planning and execution of intricate software engineering tasks” [160]. This paper demonstrates how the framework can be used to auto-generate code, build, test, and deploy from the Pull Request system, using multiple agents that communicate with each other. The process is invoked by and controlled by an end user. So far, it has been evaluated on simple tasks, with plans to extend to more complex tasks [160]. We expect to see more frameworks like AutoDev emerge that will rely on the orchestration of multiple agents as guided by interactions within the IDE or engineering system to support across both the inner and out loops of software development. These agents will also rely on programming language, development history, and the task and developer context to act as needed. It will be important to study and understand the impacts of how those agents will make the need for some interactions from developers “obsolete” (e.g., manually running tests) to introducing new interaction modes for the developers to learn and adapt to (probably through a chat interface).

2.3 The Impact of AI on Collaboration Practices in Software Development

2.3.1 Trends. Originally software developers had to work in the same room, or at least the same building, to access centralized computing hardware and its software components [137, 156, 171], as illustrated in the left of Figure 4. With the advent of distributed personal computers, collaboration and communication practices began to evolve, supporting more distributed development work [99, 137]. Video conferencing, messaging, shared repository, social media platforms, and personal devices all combined in the late 1990s and 2000s to enable much more flexible SE work practices [99]. Development tools began to integrate various distributed collaborative working processes and tools. Various messaging, video conferencing, and co-ordination tools support a variety of asynchronous and synchronous collaborations [115, 152, 171]. More recently, **Augmented Reality (AR)/Virtual Reality (VR)**, AI-powered project management and development tools, developer agents, and collaboration with bots (bot-generated content, messages, co-ordination) have led to increasing impacts on SE collaboration practices [11, 32, 110]. This includes AI-mediated collaborations, AI-support project planning and tracking, a variety of AI-powered tools, and increasingly, AI-implemented development tasks, including commit messages, comments, and notifications, as shown in Figure 4 (right) [4, 49, 127].

2.3.2 Open Challenges. A number of challenges emerge as we consider how software development collaboration practices may change in 2030 and beyond. Teams are increasingly hybrid, both in terms of distribution and with mixed human developer/AI-powered development agents [14, 35, 170]. Team and stakeholder dynamics will be increasingly impacted by this trend, as well as the trend to AI-powered agents and applications as software “end users.” It is unclear how human developers and stakeholders feel about these trends [46, 161]. The nature of future communication

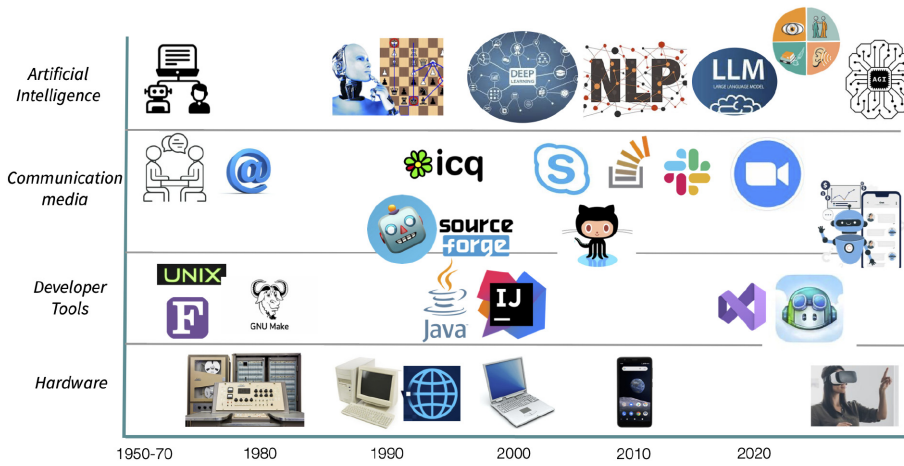


Fig. 4. Changing communication, collaboration, and co-ordination practices as hardware, platforms, and AI support evolve.

media may undergo further changes, where significant consumers of the communications are AI-powered developer agents and not humans. How trusted, or untrusted, these significantly AI-powered developer “team members” and developers are is yet to be determined [91]. How this will impact human collaboration and communication capabilities is also yet to be determined, not only in software development field but more widely in society.

Nature of “Hybrid” Teams. Software teams used to be relatively homogeneous in terms of location, tooling, responsibilities, and focus. Increasingly, teams need to include diverse human developer capabilities, and increasingly, AI-powered “team members”—developer bots or agents [45]. An SE team thus includes a mix of differently skilled and experienced humans and differently capable AI-powered agents. It is unclear what impact this is having and going to have on team climate, collaboration modes, co-ordination, and communication [40]. There is already evidence of significant changing practices around co-ordination and communication with bot-generated content, comments, commit messages, notifications, and so on [11, 127]. An interesting impact is on bot-to-bot communications. Most software development communication artifact content is still quite human-centric. Is this the right format? Is the volume appropriate for human-bot and bot-bot communication and co-ordination?

Team and Stakeholder Dynamics. AI-powered development tools and agents will impact team dynamics, but also developer-stakeholder dynamics. It is unclear how stakeholders feel about interacting with software bots, for instance, to specify defects, request changes, and others [45, 95]. Can an AI software development agent improve the interaction of stakeholders and hybrid software teams? How and when will end users interact with AI-powered developers [136]? In terms of empathy, can AI development agents “understand” diverse stakeholders, their software needs, usage contexts, and so on? Might there be opportunities for improved team-stakeholder collaboration when leveraging AI-powered developer agents? What is the “right” way to use these agents in hybrid team/stakeholder collaborations? Increasingly, AI-powered software systems will be “end users” of other software systems. What does it mean to have AI as a stakeholder? How can a hybrid team collaborate with AI-powered stakeholders?

Individual Developer and Stakeholder Perceptions. How do software engineers feel about the increasing nature of hybrid teams they work in [161]? Will they accept more and more AI-powered

tools/developer agents? Will some collaborative aspects of software development effectively disappear as AI-powered developer agents carry them out individually or collectively without human intervention [91]? How do stakeholders feel about increasingly interacting with AI-dominated development teams? Increasingly, humans have been interacting with AI-powered devices and software, for instance, smart home controllers, Siri-type assistants, increasingly sophisticated chatbots, ChatGPT-style agents, and others. Is a natural extension to request new software features and software changes directly from AI-powered developer agents, potentially with no stakeholder–human developer direct collaboration?

Communication Media. To date, most communication media used in SE—code, comments, commit messages, discussions, designs, documentation, and so on—assumes a human creator and a human reader. With increasing AI-powered developer agent generation and consumption of artifacts, including that for team communications, we may want to revisit this assumption [45]. Some SE tasks that traditionally needed a form of developer communication may be wholly or mostly AI-performed tasks [1]. AI-based developer agent communication rather than human–human or even human–AI may be better accommodated in very different ways. There may be a reduction in the need for some forms of communication. For example, some human–human tasks may become wholly or very substantially AI-performed, obviating the need for substantive associated communication media. Some forms of communication may have multiple formats—human-readable and AI-optimized. The tools used to generate/create, read/consume, summarize/translate communication-related media may of course also substantially evolve, likely being heavily AI-supported.

Trust and Understanding. In current SE teams, it is sometimes hard to tell an AI-powered bot “team member” from a human collaborator. This is likely to increase, and the distinction between different AI-powered developer agents, their capabilities and responsibilities, and those of human software engineers may further blur [104]. These synthetic developers that are hard (or impossible) to distinguish from human developers will be a part of the development team and be utilizing the same communication channels and within the same physical/virtual spaces as human SEs. How will this impact on intra- and inter-team trust [103]? Related to the previous challenge, if communication media, contents, and frequency change, how will this impact understanding of other team members, their work, their communications? The nature of SE work has already been substantially impacted by AI-powered development tools. AI-powered developer agents, their increasing deployment, the tasks, and roles they perform will all impact team dynamics, professional responsibilities, and collaboration styles.

Human Collaboration Capabilities. Humans have developed their collaboration capabilities over thousands of years. These have substantially changed in the 21st century with much greater remote work, efficiency and effectiveness supporting tools (especially AI-powered ones), evolving work places, rapid pace evolutionary demands on many software systems, and widespread adoption of new processes like DevOps [79, 115, 171]. In the realm of SE practice, developers worked originally in large rooms with substantive hardware platforms, moved to rooms with remote access terminals and personal computers, utilized open plan cubicles, worked remotely and from home, and currently often have hybrid work and collaboration contexts [87]. Each of these has brought both opportunities and challenges: freedom to work alongside others but also brought distractions; headphones and focus but with a need for controlled interruptions; synchronous video call meetings and asynchronous work, but having to manage time zones; achieving a work–life balance but at a cost of living environment conflicts; and challenges in work co-ordination with physical vs. virtual meetings. The COVID-19 pandemic greatly increased the practice of hybrid work for teams previous co-located [34]. With increasing AI-powered developer agents in teams, continued demands for hybrid and remote work, the impacts on human SE communication and collaboration capabilities

are very important to consider [54, 155]. AI-powered software project management, communication and collaboration support, and SE work support may also both support and challenge human SE abilities. Will VR and AR hardware be of any assistance (the metaverse for SE) [44, 148]? What will human stakeholders expect from software teams, both human and AI-powered virtual members? Might SE soft skills including communication and collaboration practices be enhanced, reduced, and even appraised by AI-powered tools [54]?

2.3.3 Roadmap.

- *How will increasingly common AI-powered tools and developer agents impact human–human collaboration and communication over time?* We have seen a move from collaborative spaces to headphone-dominated individual and virtual workspaces, and these have had varying impacts on human collaborative capabilities. With the increasing use of AI, we need studies on the emerging impacts on how humans collaborate/will collaborate as they increasingly use AI-powered tools and collaborate with non-human AI-powered developer agents. This may bring some collaboration benefits, but introduces dangers of reduced capacity for effective collaboration. We need to study current and future impacts of more AI-dominated SE work, collaboration between humans mediated by AI, and in overall workplaces comprising hybrid human and AI teams.
- *How will increasingly powerful AI-powered tools and developer agents, such as bots, impact human–agent and agent–agent collaboration and communication?* Communication channels may become dominated by AI-AI developer agent and/or bot communication, some channels may need optimization for these AI-powered developer agents, and some communication artifacts may fundamentally change to better accommodate such AI-powered developer agents. We need studies that explore the impact on humans of such changes, whether multiple communication channels and artifacts need to co-exist, and how AI-powered developer agents impact SE collaboration and communication needs and practices. We need studies to compare human–human and human-AI collaboration and communication patterns. We may need vastly different collaboration and communication platforms for AI-AI or AI-human collaborations.
- *Will AI-powered developer agents improve developer/stakeholder and developer/developer empathy or add barriers?* An under-researched and not well-understood area of SE is developer/stakeholder empathy—understanding each other’s different perspectives, capabilities, needs, and others, both in terms of software development but also more generally as humans. Increasing our understanding of AI-powered developer agent impact on empathy is needed, as well as increasing our understanding of the importance and role of empathy between humans in SE is needed.
- *How will increasingly powerful AI-powered tools and developer agents impact theories of distributed cognition, both human and AI-powered?* Humans and AI-powered developer agents have different capabilities, roles, communication needs, collaboration, and co-ordination strategies and will approach SE tasks with these differences. We need to develop new cognition theories about each and in combination to inform SE practices but also future human training and management and AI development and deployment.
- *Will increasing collaboration with AI-powered bots improve or decrease trust between/with humans and agents?* Many work areas have seen challenges to trust AI-generated content, AI-performed work outputs, and AI-influenced work practices. We need to study impacts on trust between humans, humans and AI-powered tools and developer agents, and potentially also between diverse AI-powered developer agents themselves.

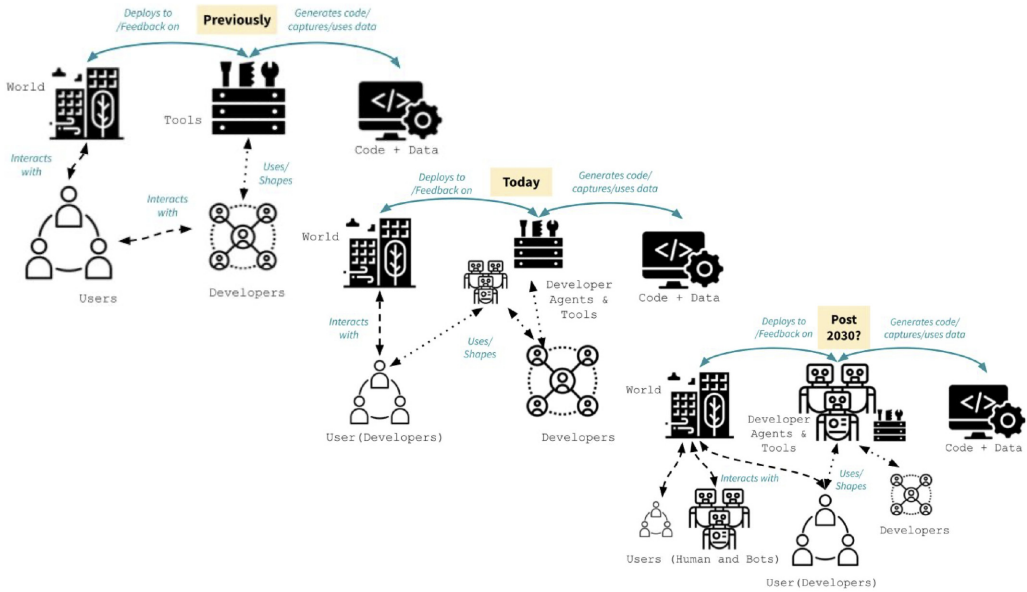


Fig. 5. Evolution of the context of software development with increasing AI-powered developer agents and AI-powered software “end users.”

2.4 Impact of AI on the Future SE Profession

2.4.1 Trends. Figure 5 (left) summarizes software development until recently. SE was the preserve of trained professionals who used complex software tools to engineer code and data used by end users (left). This significant software developer population needed substantial training and experience, attracted a professional software developer cohort, and was the intermediary between a large end user population and receiving software solutions. Many studies have looked into expectations of the profession, SE team leadership, professional ethics and responsibilities, recruitment and training, skills needs and gaps [5, 26, 68, 83, 101, 117, 149, 167]. This professionalism has the advantage of trained professionals responsible for complex safety and security critical SE. It has the disadvantages of potentially long lag times to fix defects, misunderstandings between stakeholders and software engineers, high costs, lack of sufficient engagement with diverse stakeholders, and potentially lack of specialized target domain expertise in the software team [20, 89, 126]. AI is already beginning to redefine what is SE, what does a software engineer do, and even who is a software engineer [12]. As outlined above, there have been numerous changes to software tools, productivity, and to developer communication and collaboration.

As AI-powered tools continue to be adopted, their nature and incorporation into SE practices will greatly impact SE practices and both professional and citizen software engineers [25]. As the capability of these tools continues to improve, who is able to carry out (aspects of) SE will also arguably continue to broaden. Developing ML- and AI-powered software applications themselves is inherently quite different from more “traditional” software systems engineering [146, 164]. Thus, some key trends are emerging: the advent of “AI agents” or AI-powered bots that carry out more and more traditionally human SE tasks; the advent and growth of “citizen software developers,” which is likely to result in growth in “citizen software engineers”; the greatly increasing demand for ML- and AI-powered software solutions across all domains; and redefinition and re-imagining of professional software engineer roles, recruitment, retention, and responsibilities.

2.4.2 Open Challenges. There are several significant outstanding challenges of these emerging trends in software profession. Arguably chief among them for this audience is understanding the impacts on (human) professional software engineers [94]. A second major challenge is knowing how to support emerging “citizen software engineers”—What similar but also quite different needs do they have to traditional professional engineers [12, 57, 126]? Developers are not the only role involved in software projects. Other roles include project managers, business analysts, UX designers, customer liaison, and so on. The impact of AI-powered software tools and developer agents on these roles are also unfolding at a rapid rate [53, 150]. A long-standing challenge to the SE profession has been a lack of diversity—gender, age, culture, neuro-diversity, and so on [138, 167]. Will AI-powered tools provide—as for increasingly citizen software engineer capabilities—a more diverse workforce [3]? Or will it exacerbate existing diversity issues? Or create new ones [145]? Will there be such a thing as a “professional software engineer” in a form we might recognize it in 10 years time [91]? Finally, with significant development work being done by/with AI-powered tools, that will likely further increase over time, what are the professional responsibilities going to be for (human) software engineers vs. AI-powered developer agents, especially when things go wrong [146]?

AI Software Engineers. There has been a huge increase in the range and capabilities of tools. A big change is the advent of AI “software developer” agents that are becoming an increasingly important part of software teams. These AI developers will likely take on more and more responsibilities/tasks. They will likely collaborate with SEs—and increasingly with each other—to carry out a wide range of SE tasks [45, 155]. Figure 5 (center) outlines this trend. AI-powered tools are becoming increasingly available and adopted. Developer agents, most of which are accessible to software engineers, but increasingly also to end user developers, are becoming more common place (center). Human developers still predominantly use SE tools to build software systems for end users. In the main, human developers invoke several AI-powered tools to carry out specific tasks. Increasingly, a greater range of more powerful developer agents provide support for a wider variety of SE tasks [103]. We will move more to a model of Figure 5 (right) where a greater portion of SE work is carried out by more sophisticated AI-powered “developers,” interacting with each other to carry out autonomously large SE activities. Increasingly, software will be autonomously used by other software—end user agents—that may need and expect very different software to human users. More and more end users will be “developers,” supported by these powerful developer agents. Software using agents may themselves become developer agents to autonomously evolve software systems. This trend poses a number of challenges: What tasks do we hand over/partially hand over/not hand over? How do we check quality of work done by AI developers? How do we constrain and control development done by highly autonomous developer agents? How do we protect software development tasks being done by developer agents from both deliberate attacks and accidental errors?

Citizen Software Engineers. The emergence of low-code programming tools targeted to non-professional software developers in many domains—finance, resources, science, health, education, smart cities, and so on—has resulted in the advent of the “citizen developer” [20]. These citizen developers use these low-code platforms to carry out sometimes quite detailed and substantive software development for niche domains and projects [12, 126]. These citizen developers are the domain experts, and the low-code platforms provide much faster, accessible routes to complex software solutions than employing professional software engineers to do all of the development. With the growth of AI-powered support in low-code tools, more and more complex “software engineering” tasks are becoming feasible for “non-professional” software engineers. Figure 5 (right) suggests significant growth in such “citizen software engineers,” interacting with (increasingly) powerful “AI SEs” and (less) with human SEs. The impact of these emerging AI SEs on citizen SE

work is likely to be at least, if not more, profound and pronounced than on professional SEs [57]. Many issues remain: What can be supported by the AI SEs so that technical details are hidden from the citizen SEs? When are professional human SEs needed? What do professional human SEs do? If SE bots and citizen developer tools continue to grow in uptake and usage, what is role of the citizen software engineer vs. the professional software engineer going to look like in 5-10 years time? How do SE tasks fit into some of the emerging AI tools for other disciplines, for instance, scientific discovery, materials engineering, smart cities, health research and delivery, and of course the stock market.

Other Roles. There are many other roles involved in the development and deployment of software besides just “software engineers.” These include project managers, product managers, business analysts, UX and customer teams, operations team (even if being supplanted by DevOps), documentation experts, usability engineers, help desk operators, data scientists and AI experts, domain experts, and others. The impact of AI-powered tools and processes on all of these roles will be profound, and indeed, many changes are already being witnessed. How can AI-powered project management support software development in the future [53]? What is the role of a product manager, UX designer, and business analyst when many aspects of their traditional roles can be performed, to a greater or lesser degree, with AI-powered tool support [150]? Are some of these roles going to blur with citizen software engineers? When are they a distinct role vs. a composite role when leveraging AI-powered tools? Increasingly, we see help desk and customer support roles in many fields devolved to AI chatbots. A significant amount of software documentation is being produced by AI-powered tools. Will customers expect to interact increasingly with AI-powered “helpers” in their applications?

Profession Diversity. SE teams have slowly become more diverse in terms of gender, age, location, and cognitive differences. However, team diversity is still nowhere near end user diversity, and there continue to be many efforts to diversify the workforce [138]. This includes recruiting more diverse SE students, identifying problematic team recruitment, management, and advancement practices, and identifying biases in software tools and processes. The future of AI-empowered development and development of AI-powered software brings into the question of how AI-powered tools will help enhance professional SE diversity as barriers are lowered in terms of development background [3]. As noted above, citizen or end user developers may emerge who can interact more directly with AI-powered agents or bots than those who we traditionally recruit into SE. However, AI-powered tools and AI SEs may also create new barriers to being part of the population that designs and develops software [3, 145]. This is especially a concern when we look to the future of the possibility of AI developer agents playing a critical and intensive role in software teams. Understanding the impact these tools will have on developer inclusion, productivity, experience, and well-being is of paramount concern across our industry. Issues with biased AI-powered SE tools that we currently have highlight the need to carefully address this issue [165]. There are already age, gender, and ethnicity biases in the SE workforce we want to address and not exacerbate.

Job Security and Task Responsibilities. One emergent concern of software engineers and prospective SE students is—Will I have a job in 5+ years time [91]? What SE jobs will there be and what will be their nature [94]? How will I retrain to make the best use of AI SEs and related? How are and will SE jobs be reshaped by AI-powered tools and teams [90]? It is unclear just what SE tasks will be able to be effectively given to AI-powered SEs, what kept by professional human SEs and citizen SEs, how this mix of SEs will work together, and implications on technical skills, teamwork, empathy, belonging, SE workforce diversity and inclusion, and so forth [140, 142]. Should we worry about the possibility that some existing developers that may be put out of

work? Are there training programs we should be more proactively putting into place to reskill the SE workforce?

Responsibility and Ethics. We have touched on the issue of addressing and not exacerbating SE workforce bias when adopting AI-powered tools and “team-mates” [3]. Other related issues include what are ethical uses of AI-powered tools in the workplace, including data provenance and ownership of derivative works [103]. Who is responsible if an AI SE makes a mistake leading to a critical failure? How do we reason about risks, responsibilities, audits, and so on, in the presence of significant AI-powered SE work, perhaps across large numbers of SE agents and organizations [105, 146]? What is software quality from the perspective of engineering AI-based systems with AI-based tools [67]? How do we combat adversarial SE agents in a potentially ever increasingly complex SE environment? How should future SE teams be managed—should an AI-powered SE agent be responsible for human work and welfare? How should professional ethical frameworks be amended to accommodate AI SEs?

2.4.3 Roadmap

- *What will future SEs do? Who will they work with? How will they work with AI agents? How can they best be supported by AI?* We need a variety of studies to explore emerging work practices in SE as AI-powered tools and developer agents are rolled out. This includes a rethink of traditionally human-dominated tasks, roles, responsibilities, as well as the associated collaboration, co-ordination, and communication practices as outlined in the previous section. Potentially experiments with radically different approaches to SE could shed light on the future of the SE profession.
- *What can/should citizen SEs do? Not do? How can they best be supported by AI?* A traditional danger of providing non-SE expert, non-technically trained and experienced people with tools to develop software is their inability to understand, fix, or even notice when things have gone wrong. Future AI-powered SE tools need to account for increasingly common citizen software “engineers,” providing them appropriate support for their lack of deep SE knowledge while leveraging their domain expertise. Studies identifying key areas of need, and indeed, tasks that must remain the preserve of professional software engineers, need to be identified.
- *What are future AI-powered tools for other roles?* Everyone is looking into the impact of AI on their work, since anyone even tangentially associated with software development is impacted and will be even greater impacted. Studies need to explore emerging and needed AI-powered support for non-SE roles, their impact on these roles but also on professional software engineers they must work with.
- *Who will be responsible for failures? What is ethical SE by an SE bot? Hybrid team?* Responsible SE with and for AI is critical. Studies must investigate the root cause of diverse software and software project failure and seek to mitigate these but also determine appropriate responsibilities. This includes identifying unethical, too risky, unfair, and other potential negative impacts of both AI-powered SE work and AI-powered software systems.
- *How can we anticipate and prepare for bad actors as AI is adopted in software tools?* One of the biggest risks many software companies face are threats from bad actors that also use AI-empowered tools to find novel and unexpected ways to attack software systems and steal user data. Attack vectors may be introduced at any phase of the SE lifecycle, and thus, the AI-driven SE tools of the future will need to be designed with the ability to block, identify, and mitigate threats in a way that is seamless across the entire tooling framework. Unfortunately, the use of AI that will speed up and ease much of the friction in development tasks may over time lead to new security threats, which will require a different type of automation to address

them, forcing the design of new tools to help manage this new friction. Anticipating what AI in SE tools may “flip” or “reverse” into may help us recognize the risks and preemptively design tools to mitigate these concerns.

2.5 Impact on SE Education

2.5.1 Trends. We have built up over several decades the foundations of a software profession, including initial education and on the job training. The *Software Body of Knowledge (SWEBOK)* has been refined over many years by *ACM* and *IEEE* to try and guide us in ensuring well-trained software engineers are available to society [22]. We have used this body of knowledge to construct higher education and workforce training for professional software engineers and to better understand their skills gaps and how to address them [59]. Our curricula and body of knowledge have undergone periodic updates to incorporate a number of technical and professional changes. These include, for instance, adoption of agile methods, cloud platforms, DevOps, security and privacy, codifying various professional responsibilities, and incorporation of more team- and project-based learning approaches.

2.5.2 Open Challenges. A number of professional education challenges are present. What should the SWEBOK contain now? In 5 years time? What does the emergence of AI-powered software tools, citizen software engineers, developer software agents, and AI-powered software “end users” mean for software education [88, 106]? What are the future skills needed by professional software engineers, citizen software engineers, other roles involved in SE, and indeed software development agents [30, 53, 94, 150]?

SWEBOK. It has been a challenge to keep the software body of knowledge up to date as the technical, professional, and societal demands on the profession have changed. As discussed in the previous subsections, it is difficult to foresee just what the change in software engineer tools, collaborations, professional skills and expectations, and roles will actually be. With the advent and increase in AI-powered software tools, just what skills does a software engineer of the near and more distant future actually need? It may be that new processes, tools, and techniques, supported by AI-powered tools and developer agents, substantially change the nature of SE and mean that some traditional skills are not required, or much less commonly required. The ability of domain experts and end users—citizen software engineers—to develop substantial aspects of software systems may mean that the number and nature of skills of professional software engineers substantially change. Should there be a SWEBOK for citizen software engineers? For AI-powered developer software agents? It seems essential for SE students to gain knowledge of engineering AI-powered software systems, including the various technologies involved [29, 140, 142].

Training for Work in Hybrid Development Teams. Software engineers increasingly work with AI-powered tools, collections of AI-powered tools, and developer agents. They will continue to work with humans, including other software engineers, other development-related roles, and end users and stakeholders. However, the expectations, tasks, and roles of both other humans and AI-powered agents and tools may greatly change. It is unclear how a hybrid development team of humans and AI-powered developer agents should be formed, organized, managed, tracked, and so on. It is unclear what the impact on human software engineers will be of their changing roles and expectations, heavy use of AI-powered tools and collaboration with AI-powered software agents, hybrid and remote work, and how changing roles of other humans involved in development may impact professional software engineers [27]. At the least, it seems essential that SE students will need to gain significant experience and skills with choosing, configuring, using, and assessing a wide variety of current and emerging AI-powered tools and “team mates” [29, 140]. They also need

experience in building a variety of ML- and AI-powered systems [85, 164] and carefully addressing ethical and responsible AI issues [129]. They may also need to gain a better understanding and experience with changing expectations of their stakeholders who themselves are becoming much more familiar with and much heavier users of AI-powered software in their own work domains. AI-mediated stakeholder interaction seems likely to continue to increase. As noted in earlier subsections, AI-mediated developer interaction also seems likely to continue to increase. Training and experience in such environments is likely to be essential for future software engineers.

SE Education Delivery. Like all other educators, SE educators are carefully considering how to leverage AI in their teaching and assessment, both in a higher education setting and in the workplace [132]. AI-powered tutors and assessors have been experimented with in many domains. Many outstanding issues remain, but it seems highly likely that SE education delivery itself will be highly impacted by AI-powered tools [52]. Some challenges we are familiar with, for instance, did the student write their code/tests/design or was it copied/adapted (or produced by generative AI)? The SE and CS education community has deployed various AI-powered tools including code and other software artifact plagiarism detectors for many years. Many SE programs have been experimenting with incorporating generative AI and other techniques into curricula [37, 52, 75, 132]. Thus, deploying AI-powered software tools and developer agents during software engineer training does not seem a big stretch. However, assessment practices are likely to need a significant rethinking. Software engineers have been heavy adopters of generative AI in supporting on the job training, technical question answering, generation of tutorials, and other supporting materials for learning.

Attracting and Retaining Future Software Engineers. Given the dramatic changes likely in the SE profession, including much traditional work being taken up by AI-powered tools and developer agents, many are wondering just what the future profession will look like—and indeed if there will be one [91]. Increasing questions at University open days around what the future careers in SE will look like, what career trajectory they will bring, and whether it is going to be a good career choice demand responses. It may be that many aspects of SE that attracted people to the professional no longer exist or are massively changed in the near, let alone distant, future. It may be that a very different set of skills and aptitude will make for a “good software engineer” of the future [101]. How will we “sell” the prospect of SE and being a software engineer? Will there be vastly different pathways into SE and career pathways within the profession? Will the need for the traditional concept of the professional software engineer increase, decrease, or remain largely the same with the continued development of AI-powered developer agents? Will domain knowledge be far more important than SE skills and knowledge, with much of SE undertaken by developer agents under the instruction of end users and citizen software engineers? Or will the increasingly complexity, range, and capability of software systems necessitate a continued substantive profession?

2.5.3 Roadmap.

- *What should be in the Software Body of Knowledge in 2030?* We need to engage with practitioners and educators to reformulate the SWEBOK cognizant of the big impact AI-powered tools and developer agents are having in SE work and education. The degree of change in emphasis on topics in SE curricula and for different future SE roles also needs careful consideration.
- *What does an SE curriculum and an SE career of the future look like?* A danger of all education curricula is backward-looking or current skill demand-driven design, instead of future likely need. Educating all future workers for better awareness and skill in working with emerging AI-powered technologies is essential, including their current and likely limitations. Significant experience with diverse current and emerging AI-powered tools when educating software

engineers seems critical. Is the curriculum similar or quite different for a “citizen software engineer” to that of a professional software engineer curricula? The needs of non-professional citizen software engineers must not be ignored, and their education may need to be quite different from future professional software engineers. For start, they are likely to need a primary domain-specific education with engineering software leveraging this domain expertise a secondary focus, however, well integrated. On the job, citizen SE training (perhaps significantly AI-enabled) may be more viable in many instances.

- *How do we effectively deliver a future SE curricula, including leveraging AI educational tools and providing experiential learning and practice with AI SEs?* While SE education has always had to be a degree process, technique, and especially tool agnostic—all organizations have greater or lesser degree of variation in each of these—providing knowledge, training, and experience with some practical combination of these has been a feature of almost all SE degree programs for many years. However, many software engineers have gained significant skill enhancement via tutorials, workshops, video tutorials, online communities, and—increasingly—AI-supported guidance and support. It seems likely that further AI-supported education and training studies are needed, including emerging AI-supported approaches to delivery and assessment.
- *How do we attract and retain excellent software engineers of the future?* There are promising and threatening aspects to greater use of AI-powered tools and emergence of AI-powered developer agents. Studies need to include a diversity, inclusion, and equity aspect to adopting AI-powered SE. This includes addressing bias and unfairness in AI training, diverse usage practices, diverse human SE needs and expectations, and carefully assessing negative consequences or potential consequences of adopting varied AI-powered solutions. It also remains to be seen if the emergence of AI-powered tools that significantly change SE work and SE job expectations enables many who previously did not embark on an SE career to do so. Proactive efforts to present varied SE work of the present and of the likely future, engagement with potential future students and professionals, and increasingly engagement with likely future citizen software engineers and part-time professional software engineers are needed. Ultimately, we need to include risk assessments of AI-powered solutions from individual, team, organizational, and societal perspectives.

3 Human-Centric Software Systems in the Era of AI

Software systems are inherently designed for humans, who interact with them either directly, such as with software applications, or indirectly, as with embedded systems. Over the years, advancements in technology have transformed these interactions. From mice and GUIs in the 1970s to the Internet and web in the 1980s, mobile devices and Wi-Fi in the 1990s, and cloud computing at the turn of the century, technological shifts have continuously redefined HCI. More recently, XR, wearable devices, and generative AI exemplify the innovations driving these changes.

The rapid evolution of XR and generative AI is fundamentally altering human–software interactions. The blurring lines between VR and actual reality, coupled with the rise of voice, video, and gesture-based interactions, are gradually replacing traditional GUI-based interfaces. Generative AI further amplifies this shift, reshaping the role of humans from *passive users* to *active participants* in smart, human-centric systems and ecosystems. These ecosystems range from smart living applications, such as smart homes, smart grids, smart transportation systems, to large heterogeneous *human-centric ecosystems* that emerge from the coexistence of systems and humans in complex environments.

However, human diversity—encompassing personality, emotions, ethics, culture, age, and gender—significantly influences how individuals interact with AI-powered systems. These interactions,



Fig. 6. The disruptive effects of human-centric systems.

enabled by generative AI, XR, and multimodal communication methods, redefine not only software design but also the way humans live and function within software ecosystems.

We define *human-centered systems* as software systems designed and operated based on human values and needs. These systems, often highly automated and AI-driven, aim to balance automation with the preservation of human control. They strive to deliver ethical, efficient, and inclusive outcomes, maximizing the benefit to humanity. As illustrated in Figure 6, McLuhan's Tetrad (introduced in Section 1) offers a framework to analyze the disruptive impact of these systems on SE by examining what they enhance, retrieve, reverse, and obsolesce.

Human-centric systems *enhance* diversification, customization, evolution, and adaptation, and *obsolesce* rigid and static operations. They enhance human diversity, heterogeneity, evolutionary, self-adaptive and autonomic SE activities, field testing, autonomous bots, and data productization. They *obsolesce* rigid and recurrent engineering activities, like structural and regression testing, the **Monitoring, Analysis, Planning and Execution (MAPE)** cycle, and component-based architectures. Human-centric systems *reverse* HCIs and *retrieve* requirement engineering and dynamic analysis. They reverse natural language, image and video processing, generative AI, and XR, which become the cornerstones of the new role of humans in the system. They retrieve runtime simulation and digital twins to reason about seamlessly evolving behavior that emerge from evolving human interactions. At the same time, they *retrieve* models at runtime, dynamic and incremental analysis, to reason about evolving behavior that derive from unconstrained interactions, and domain-specific languages, requirement engineering, and enterprise architectures to drive generative AI toward acceptable solutions.

The four diamonds of Figure 6 outline what smart, human-centric ecosystems enhance, retrieve, reverse, and obsolesce. The **ENHANCES** diamond indicates what smart ecosystems largely amplify: diversity and customization, self-adaptation and evolution, field testing and autonomous bots, autonomic verification, human models, and data productization. Customized services and systems have been studied in the past decades. Software product lines [31] and APIs [97] are two opposite ways of customizing software systems by tailoring general frameworks to specific needs (product lines) and opening systems interactions toward extensible frameworks (APIs). Generative AI and XR lead to ecosystems that autonomously emerge, evolve, and adapt over time toward ultra large, smart, human-centric ecosystems. Self-adaptive and evolving systems have been widely studied since the beginning of the century [58] to cope with the lack of central control and specifications of Ultra-Large Software Systems. The intrinsically evolving nature of smart systems and ecosystems

emphasizes the need of self-adaptive approaches to cope with emergent behaviors and sometimes unavoidable failures. Field testing [19], self-adaptive systems [33], autonomic verification, and more recently infrastructure testing and test bots [45] move testing and verification from testbed to productization environments and pave the way to a completely new way of verifying adaptive and evolving smart, human-centric ecosystems.

The RETRIEVES diamond highlights technologies that have been widely studied and used in the past and that will find a new role in smart, human-centric ecosystems. Simulation [13] and digital twins [144] are commonly used to train humans with safety-critical systems and test complex cyberphysical systems. Testing the evolving and adaptive behavior of smart, human-centric systems requires complete and consistent digital twins to capture the all and only aspect of the real system that are necessary to observe the behavior of the real system. Formal models, static and dynamic analysis, have been studied for many decades with many successful applications in safety-critical systems. Smart, human-centric ecosystems will retrieve dynamic and incremental analysis to cope with their evolving and adaptive behavior. Smart, human-centric ecosystems will retrieve domain-specific languages [111] and enterprise architectures [139] to cope with the dynamic complexity of the systems.

The REVERSES diamond indicates the technologies that smart, human-centric ecosystems will flip into when pushed to the extremes: natural language, voice, image and video processing, and more generally generative AI and XR, all of which are the fuel of the future smart ecosystems.

The OBSOLESCEs diamond summarizes some popular engineering approaches that cannot cope with the new characteristics of smart systems and ecosystems: structural, regression, and GUI testing, the MAPE autonomic cycle, and in general self-adaptive cycles. Structural, regression, and in general common testing approaches, including GUI testing, assume both the repeatability of the executions of the target systems executed on testbed platforms and the availability of some specifications in the form of test oracles to validate the results. GUI testing assumes that users interact with the system only through the GUI. Smart, human-centric ecosystems evolve over time, adapt to emerging scenarios, and depend on unconstrained human behaviors that cannot be fully reproduced on the testbed.

The subsequent sections examine the impact of emerging technologies on human–system interactions. These technologies, which drive the development of human-centric systems, enable continuously adaptive software that challenges traditional UI design, fosters the spontaneous emergence of large ecosystems from the implicit interactions of independently managed systems, and reshapes development and operational practices. In particular, Section 3.1 explores the evolution from traditional GUIs to intelligent **Adaptive User Interfaces (AUIs)**, Section 3.2 addresses the challenges of designing and verifying large, emergent ecosystems, and Section 3.3 examines the shift from **No-Operations (NoOps)** to **AI for Operations (AIOps)** in development and operational activities. Together, these discussions highlight how new technologies are driving the creation of continuously adaptive, human-centric systems that redefine the interaction between humans and software systems.

3.1 From GUI to Intelligent AUI

3.1.1 Trends. The demand for efficient and user-friendly software systems is rapidly increasing. Modern software systems must continuously adapt to the unique characteristics of diverse users, platforms, and environments. This shift forces developers to embrace change as an integral part of the development process. The notion of *software adaptation*, which currently focuses primarily on system-level changes, is evolving toward *intelligent adaptation*, where AI, software systems, and humans collaborate to continuously adapt and evolve human-centric ecosystems in real time, enhancing their quality and **User Experience (UX)**. In this context, we envision humans and

AI-enabled systems collaborating to achieve common adaptation goals over time, *evolve* together, *learn* from each other, and collectively *improve*.

Since diverse humans are involved, we need *intelligent adaptations*, where human-centric smart ecosystems can learn to adapt themselves to different users based on the characteristics of the context of use. This includes characteristics of the *human being*, such as emotion, technical proficiency, gender, personality, and physical and mental impairments; the *platform*, such as Web, desktop, tablet, phone, and mobile applications; and the *environment*, such as location, mobile conditions, light, noise level, physical configuration, organizational and psycho-social constraints, as well as available interaction data. Self-adaptive systems, model-driven engineering, and AI are key enabling technologies to support such intelligent adaptations, although significant challenges remain.

3.1.2 Open Challenges. Humans come from diverse backgrounds with varying expertise, cognitive, and physical traits. Their needs and preferences may change over time, requiring software systems to identify new scenarios and adapt appropriately without costly re-engineering. AUIs can address diverse user needs by customizing the UI to meet individual needs, goals, and context of use [120, 143]. While the concept of AUI is not new, advancements in AI techniques bring new perspectives on its realization. Here, we analyze the current status of this concept and highlight the main open challenges.

UI Adaptation in HCI and SE. An effective AUI must apply the right adaptation at the right time in the right place to maximize its value for humans. UI adaptation typically involves a three-stage process: *perception* (recognizing and interpreting the current context), *decision* (determining whether an adaptation is needed and what action to take), and *action* (implementing the adaptation). This process can radically change when a third party, such as an AI agent, supports the adaptation process (i.e., to recommend or perform UI changes during user interactions with the system).

Over the last three decades, the SE [6–8, 166, 172, 173] and the HCI [62, 63, 98, 112, 157, 162, 175] communities have made substantial progress in AUIs’ design and evolution, particularly leveraging AI techniques. However, emerging technologies, especially generative AI, introduce new challenges and opportunities. The HCI community has proposed numerous methods for UI adaptation, but most focus on the adaptation of individual UI elements (e.g., menus) [65, 157, 162]. There is a pressing need for holistic approaches that address the adaptation of the entire UI in the context of human-centric smart ecosystems. The emerging field of **Intelligent User Interfaces (IUIs)** explores the interplay between AI and HCI. While many IUI techniques focus on improving communication between users and systems through advanced interaction methods—such as natural language processing, gaze tracking, and facial recognition—not all IUIs have true learning or problem-solving capabilities. The SE community has introduced principles and technologies to support UI adaptation at the system side (e.g., MAPE-K adaptation loop, models at runtime), but often relegates the perception, decision, and action stages of adaptation on the end user side, typically addressed in HCI, to a secondary role.

Limitations of Current AUI Approaches. Despite some promising empirical studies [39, 55, 159, 175], current AUI approaches remain unpredictable (users do not know when and how adaptations will take place) [56], choose detrimental adaptations with unacceptable frequency, incorrectly capture user needs, cause cognitive disruption (users are disrupted by the adaptation) [100], lack user involvement (users cannot actively participate in the adaptation process), and lack explanation (users are not informed of the reasons for adaptation).

Adaptive strategies for AUIs open many challenges to effectively leverage learning capabilities: How to select an adaptation strategy? How to model human factors? How to model AUIs at runtime? How to coordinate human and technical aspects? How to handle heterogeneous input

data? How to predict UX with AUIs? How to support decision-making? How to support AUIs in specific environments such as XR?

Selecting Adaptations. A key technical challenge of AUIs is how to select adaptations and design an effective AUI in the presence of a huge design space of AUIs [162]. A system must decide *what* to adapt, and *when* or *when* not to change. An important issue is to determine the *sequence of UI adaptations* to be carried out to maximize the user benefits and reduce the costs, for instance, improve the system quality, improve the user performance or experience, and reduce the workload.

Several studies have applied different approaches: heuristics, rules, and ML methods (supervised learning, reinforcement learning) to support the adaptation process. *Heuristic-based* approaches adapt specific UI elements [38] and work well when the sensed input is highly predictive of the most appropriate adaptation, for example, in the context of menu-based interaction, heuristics that exploit click frequency, visit duration, or other metrics obtained from interaction data are frequently used. *Rule-based* approaches modify the UI behavior based on pre-defined rules, like, event-condition-action rules [8]. They transparently monitor the executed adaptation actions and thus facilitate the modification of the adaptation process; however, they do not support emerging behaviors and require experts to specify the user goals. *Predictive ML-based* approaches learn how to adapt AUIs. *Supervised learning approaches* work well when the user state is highly predictive of the appropriate adaptation, for instance, a mapping is learned between user data and suitable adaptations, but require deep training with high-quality datasets that describe the consequences of possible adaptations on potential users. *Reinforcement learning approaches* formulate the UI adaptation process as a stochastic sequential decision problem and train agents based on some rewards received from the environment for certain actions. They require an extensive amount of poor attempts to learn a good policy, and this is a strong limitation for large state-action spaces, such as those needed to design AUIs for human-centric ecosystems. *Model-based* and *multi-agent* reinforcement learning are promising approaches to address these limitations. Model-based reinforcement learning approach by Todi et al. uses HCI models to predict rewards for each state during simulations [157]. An HCI model is a computational model that explain how users interact with interfaces at the level of individual human cognition [130]. Multi-agent reinforcement learning approach by Langerak et al. pairs a user agent that mimics a real user and learns how to interact with a UI [98] with an interface agent that learns UI adaptations to maximize the user agent's performance. Model-based and multi-agent reinforcement learning indicate interesting research directions that need to address their limited scalability and an effective way to determine the rewards [60].

Designing Intelligent AUIs. The design of intelligent AUIs raises several key challenges about how to structure software and AI components to support intelligent UI adaptation, incorporate human feedback when training AI models to support UI adaptation, and support the human-centric adaptation. Abrahão et al. [2] recently proposed a conceptual reference framework to support the model-based IUI adaptation, but we still need to experiment with different AI techniques in different contexts of use to structure software and AI components to support intelligent UI adaptation. The current approaches based on predictive HCI models to incorporate human feedback when training AI models do not take into account changes in human behavior that may occur during user interaction with the system [157].

Modeling Human Factors. Many approaches rely on some user models to support the adaptation of human-centric ecosystems. However, structure and usage of the current user models are not sufficiently explicit to help developers model human factors and implement UI adaptations. There is a distinguished body of work looking at how best to take users' existing practices into account when

designing software. These include consideration of the social and organizational context, designing software where there is a cognitive fit between users' mental models and software representations, and acknowledging that users have cognitive biases that will affect system adoption. Participatory Design and Co-design [141] have tackled these challenges head-on by including users and other stakeholders as equal partners from the very beginning of the design process. However, the research on fully and descriptively modeling the users has not produced fully satisfactory results so far. It is also unclear how models representing human factors can be used to drive UI adaptations. Therefore, modeling and design of human factors are essential steps toward designing and adapting human-centric ecosystems.

Runtime Models of AUIs: Models at runtime [21] have been successfully used to automatically reflect changes from a system into changes in models, and vice versa, in several domains. The availability of configurable runtime models makes it possible to use them as interfaces for monitoring and adapting AUIs, and to test adaptations at the level of models before actually adapting the running system [33]. However, the traditional adaptation in which the variability is fully pre-defined does not adequately address human diversity. We need smart UIs that can learn how to adapt to different users based on a user model that captures the user preferences, behavior, style of interaction, expertise, emotions, and so on.

Diversity in Input User Data. We can decide how to adapt UIs based on a variety of sources including empirical data, end user feedback, and biometric sensors that provide essential data to recognize users' physiological states during the interaction in various scenarios. For example, we can detect emotions by processing data from electroencephalography sensors [61, 64]. We can measure stress and arousal with data from the skin conductivity monitored with galvanic skin response sensors [123]. We can measure stress levels and muscle-arousing activities with electromyographic sensors [70]. We see many different physical signals that we can use to support UI adaptation more effectively. However, how can AI agents learn from these data to recommend more meaningful UI adaptations is a largely open issue.

Predicting UX with AUIs. We can measure some aspects of UX by analyzing physiological measures that represent different human affective states (for instance, pleasure, stress, relaxation) when performing a particular task. These kinds of measures have received increasing attention from the SE community in the last few years. A recent systematic literature review on the use of physiological measures in SE [168] revealed several empirical studies on measuring the cognitive load in SE activities, such as code comprehension, code inspection, programming, and bug fixing. However, there are only few experiments on understanding end user's emotions and feelings when interacting with UIs and human-centric ecosystems [61]. We can measure and monitor end user emotions and feelings during user interaction to improve the UX.

Tradeoffs in decision-making. There is a lack of decision-making processes that support tradeoffs between system quality attributes, user characteristics, and adaptation rules. We need guidelines to define and operationalize decision-making methodologies and to guide decisions on the use of AI inferences (for instance, exploring different ML algorithms) instead of simply waiting for user input.

AUIs in XR Environments. XR is a disruptive computing platform that merges VR, AR, and mixed reality toward computing environments where end users seamlessly interact with both the real and virtual world, with great potential to a variety of sectors, for instance, entertainment, education, medicine, and manufacturing industry. However, we still have little experience on designing AUIs that present interactive content to end users in XR immersive environments. XR requires a UI that

overlays virtual content onto or embeds it within the real, physical environment without costs to end users' attention, effort, and safety [158].

3.1.3 2030 Roadmap.

- *How can human diversity and different human needs drive UI adaptation?* Humans are very diverse in needs, preferences, values, gender, age, ethnicity, culture, language, educational attainment, socio-economic status, living and work conditions, personality, motivation, emotions, behavior, cognitive challenges, physical challenges, and so on. Each human is unique and has unique needs, preferences, and ways of interacting. The human differences dramatically impact on the design and adaption of AUIs: How do we model an infinitely large space of diverse characteristics, and use them to drive UI adaptations? What are the best abstractions to effectively represent the human characteristics? What is the most suitable theory to capture them? Digital twins and domain-specific languages offer great potential to represent and simulate human characteristics and their interaction, but we still need to formalize these diverse human characteristics and their impact on AUIs, by leveraging and/or adapting existing theories in fields such as social sciences, behavioral sciences, cognitive psychology, and so on. The area of individual differences in psychology has a long track record of validated measures of stable individual traits over time (see, for instance, [131]). From a more practical viewpoint, a theory for human diversity in AUIs can give us input for decision-making regarding choices of UI adaptations and their impact on user characteristics and system quality.
- *How will AI-powered systems (AI agents) interact effectively with humans?* We need precise design guidelines for human-AI interactions. Recently, 18 generally applicable design guidelines for human-AI interaction have been proposed [9, 102]. Examples of such guidelines include: make clear what the system can do; show contextually relevant information; learn from user behavior and update and adapt cautiously. While these guidelines provide a starting point for understanding human-AI interactions, it is not clear how they can be operationalized and used to design intelligent AUIs. How can the system learn from user interactions with the system and AI agents to recommend meaningful adaptations? AUIs should integrate adaptation capabilities and computational learning. Adaptation could be accomplished by using the knowledge stored in a User Model and inferring new knowledge using the current user interaction data. Computational learning and models at runtime can be used when knowledge stored in the User Model is changed to reflect new situations or runtime data. However, how do we represent a User Model, use it at runtime, and evolve this model to reflect the changes in the user needs and preferences?
- *How can we design explainable AUIs?* A large body of work exists and continues to grow on how to increase transparency or explain the behaviors of AI systems (for instance, [10, 92, 134]). However, we need techniques and mechanisms to explain the underlying rationale of an intelligent AUI to the end user in a logical and understandable way. The main challenges in explaining UI adaptations made by AI agents are: Shall we explain the individual decision or the whole policy? How can we actively measure users' understanding of the agent's decision process? How to present the explanations to end users (for instance, white box vs. black box explanations)? How to get users' feedback to continuously improve the explanation process? How to provide users with easy controls over the output of the intelligent AUI/ML algorithms?
- *How can we create an accurate mental model of the ML algorithms underlying AUIs?* We need transparent AUIs with respect to the user's actions that can contribute to the output of the adaptation algorithm. We need to understand how people form mental models of the way the AI agent works, given different types of explainable AI explanations, and the cognitive

loads they incur in forming these mental models. Few recent studies started investigating this problem [10]; however, we still do not understand how to gather empirical evidence for designing explainable AUIs?

- *How can we design AUIs that adhere to fair-by-design principles?* Users may experience unexpected or confusing information that violates the expectations and hints at algorithmic bias, while interacting with the system [47]. How can we handle bias in the UI adaptation process? How can we detect bias in the datasets or decision-making process supported by AI agents, for example, wrong decisions to adapt the UI taking into account the user's gender or membership of marginalized social groups (race, health, disadvantaged socio-economic background)? UI designers need specific training and techniques to recognize social norms and biases. GenderMag [24], a method for identifying gender biases in UIs, only scratches the surface of a huge problem that need further investigation. We need tools for assess and repair fairness; we need controlled experimentation environments to promote awareness of fairness during the AUI design, and effectively test and improve bias-related issues before releasing an intelligent adaptation strategy for AUIs. The AI Act, a proposal for a European regulation on AI released in April 2021, defines the concept of *sandboxes*: controlled environments that facilitate the development, testing, and validation of innovative AI systems for a limited time before their placement on the market, a first step in this direction.

3.2 From Systems-of-Systems to Human-Centric Smart Ecosystems

3.2.1 Trends. The key role of humans in software-based systems has been acknowledged and studied since the early 2000. Common SE systems constrain human–system interactions within well-defined interfaces that restrict the human actions that software engineers consider in the software development process. The software process models assume the existence of requirements to elicit and analyze, and indicate how to model and implement the system architecture, and how to verify and validate the system with respect to its requirements. Common approaches to design and validate software systems consider humans as mere *users* of the system and rely primarily on system models, thus largely ignoring the impact on human attitude on the ecosystem. Northrop et al. recognized the new role of humans in **Ultra-Large Systems (ULS)** already at the beginning of the century (“*People will not just be users of a ULS system; they will be elements of the system*” [121]). More recently, Grundy has emphasized the role of humans in *human-centric systems* that heavily rely on the *use of Internet of Things–type sensors, interactors, controllers, ... capture and use diverse types and amounts of data, ... and ... increasingly require some hybrid form of cloud- and edge-computing*, and stresses the impact of *human-centric issues* [72].

The technological revolution of the last two decades and the recent advances in generative AI, AR, and the Internet of Things have a disruptive impact on human-centric systems. El Moussa et al. characterize human-centric systems that heavily rely on generative AI as *smart ecosystems, multi-faceted systems that emerge from the composition of independently operated and autonomous systems with smart functionalities and that evolve over time, as in the case of next-generation smart cities, smart buildings, smart grids, and more generally a future smart planet* [42]. Smart, human-centric ecosystems, like smart cities and smart grids, evolve and adapt over time to match to the infinitely many human activities and needs that heavily impact on the ecosystem. New types of independently operated smart systems, like autonomous vehicles and mobile devices, constantly enter and exit the ecosystem. Generative AI-centered systems learn from the ecosystem and adapt their behavior accordingly. The human behavior changes and evolves within a smart, human-centric ecosystem and determines new quality attributes. While classic systems operate according to well-understood requirements that determine the correctness of the system, *smart systems*, like bots and autonomous vehicles,

challenge design and verification with adaptive behaviors that emerge and evolve over time to adapt to different human behaviors. *Smart, human-centric ecosystems*, like smart grids and smart cities, are not owned, specified, and maintained by a single owner. The city authority regulates the behavior of the different systems within the city; however, the city authority does not own the many systems that coexist in the city and that evolve according to specific and sometime intrinsically contradicting requirements [41].

3.2.2 Open Challenges. Many open challenges emerge when trying to engineer future smart, human-centric ecosystems. Traditional ideas of correctness need to be revisited. Humans are diverse and interactions may be unpredictable, unconstrained, and behaviors unanticipated, especially as systems become more self-adaptive. Traditional SE methods, including testing, analysis, and monitoring, all struggle with aspects of smart ecosystems engineering. Self-adaptive systems are likely to grow significantly. It is unclear how humans impact such self-adaptation and how various AI-powered solutions will support diverse self-adaptive smart ecosystems.

Correctness of a Smart, Human-Centric Ecosystem. The concept of correctness—defined as compliance with a specification—does not sufficiently capture the quality aspects of a smart, human-centric ecosystem. The behavior of a smart city can obey the regulations and can well suit the needs of the humans in the city; however, it is not simply correct or wrong. We may witness a major degradation of a smart city, but we do not have the elements to define a failure as a deviation from the specifications, in the absence of complete and agreed specifications of the ecosystem and in the presence of emerging and sometime contradicting behaviors of the systems in the ecosystem. For example, we can observe a major degradation of the smart city due to unexpected and implicit interactions among the traffic control system that reduces the access to the highway to avoid traffic jam, the pollution control system that reduces the speed limits to lower the pollution, and the heterogeneous and unpredictable behavior of the autonomous and non-autonomous vehicles. El Moussa et al. propose a new concept of *healthiness* to capture the acceptable behavior of a smart ecosystem that captures the strength of the ecosystem, the ability to exhibit a stable behavior, and the capability to react to unavoidable crises [42], a preliminary step toward a new concept of quality of smart human-centric ecosystems.

Unconstrained Human–Ecosystem Interactions: Unconstrained human–system interactions within smart, human-centric ecosystems upset the SE landscape. Software engineers cannot engineer smart ecosystems by limiting their attention to the system interfaces while ignoring the human attitude. Autonomous vehicles cannot simply return the control to the drivers in the case of unexpected scenarios. SE shall take into account the infinitely many possible responses and reaction time of drivers with different attitude, personality, and expertise. When engineering smart ecosystems, software engineers cannot simply rely on an expected uniform human behavior according to regulations. They must consider the different behavior of diverse humans with different motivations, personalities, emotions, languages, and more.

The *greedy crash scenario* presented in [42] is a good example of how human motivations and personality may lead to a smart ecosystem failure also when the systems that comprise the ecosystem behave correctly: An unexpected event, like a fire alarm in a busy underground station, may cause a high peak of requests of both taxi and Uber rides in the smart city ecosystem. Uber raises the fares in the presence of unbalanced requests and available drivers, and the tests can verify that the ecosystem restores the usual request-availability balance with common pricing after a relatively short perturbation, when humans behave as expected. However, some drivers, the *greedy drivers* as called in [42], may decline the requests by expecting further increases of fares. A few greedy drivers may not impact the ecosystem; however, a test suite that takes into account also

the human behavior, and in particular possible *greedy behavior*, can reveal that an unpredictably large percentage of greedy drivers can lead to an explosion of fares and a consequent failure of the ecosystem, even if all systems that comprise the ecosystem behave correctly [42]. A *greedy behavior* can depend on motivations and personality. Conscientious drivers, who rely on Uber as their main income, hardly decline requests, while overconfident drivers, who rely on Uber for some extra income, may exhibit a completely different behavior.

In general, approaches to assess the quality of smart cities must take into account the attitude and personality of humans and social groups that may lead to different behaviors and change over time due to unexpected implicit interactions of smart systems with the smart city. Human behavior has been widely studied for centuries, and there are many modes of different aspects of human behavior. The *Big Five personality* traits, also known as the *OCEAN* model [124, 125], that behavioral psychologists have developed over the past decades well captures the human personality and can greatly help software engineers understand the complex intertwining among human actions that stem from different personality and characteristics, to define suitable models for engineering smart, human centric ecosystems.

Self-adaptive Smart Ecosystems. Smart systems, like self-driving cars, rely on deep learning engines, the behavior of which depends on the training data and non-deterministically evolves over time. As such, smart ecosystems cannot be either completely specified in advance or deployed on a testbed platform, and their behavior is seldom repeatable. Humans are active elements of smart ecosystems and their interactions cannot be framed with UIs. The classic MAPE autonomic cycle, and in general traditional self-adaptive cycles, monitor the autonomic system and execute corrective actions. The main approaches defined in the past decades follow the *closed-loop approach* postulated by Müller et al. “*Architectures based on closed-loop feedback control offer appropriate capabilities to address this challenge*” [116] and focus on the cyberphysical system. Smart ecosystems include humans as key elements that escape monitoring and executions in the forms required for MAPE self-adaptive cycles. The next generation of self-adaptive approaches cannot ignore humans. We need new self-adaptive approaches that include models of both the systems and the humans, who comprise the ecosystem. Smart, human-centric ecosystems enhance self-adaptive and evolving systems with models of interdependence of human actions due to human characteristics. New models of human behavior will complement autonomic verification to verify the complex intertwining between the smart systems and ecosystems and the humans.

AI-Powered Testing. Most testing approaches execute test cases on testbeds, before deployment. Static analysis approaches check code and artifacts. Dynamic program analysis work on execution traces. Smart, human-centric ecosystems emerge and evolve over time with continuously updating training of AI-powered systems, and systems as well as humans continuously entering and leaving the smart, human-centric ecosystem. The intrinsic evolving and adaptive nature of smart ecosystems makes it impossible to fully test the ecosystem on testbed. Field testing [19] and more recently test bots [45] have been studied since the beginning of the century to cope with failures in production. Both the *in vivo* and *in vitro* approaches developed so far aim to test software systems with the data from the production environment and target mostly unit testing. Smart, human-centric ecosystems enhance field testing as the primary way to both assess the quality of evolving behavior and capture emerging failure conditions before major impacts on the ecosystem. Autonomic verification, that is, verification of the smart system and ecosystem behavior while emerging and evolving over time, will enhance the early approaches of field testing and test bots, by merging field testing and dynamic analysis approaches with models of human and social behavior to verify the evolving behavior of the systems.

3.2.3 Roadmap

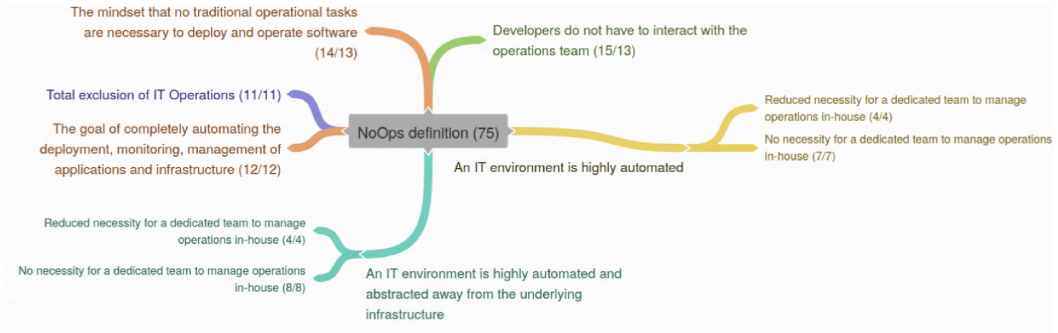
- *What is correctness for a smart, human-centric ecosystem?* We need to work out new ways to characterize and formalize the quality of a smart, human-centric ecosystems, in the absence of complete and agreed specifications of the ecosystem. We also need to do this in the presence of emerging, adaptive, and sometimes contradicting behaviors of the systems in the ecosystem.
- *How do we handle unconstrained and implicit interactions among humans and systems that comprise a smart, human-centric ecosystem?* We need to develop new ways to predict, analyze, and test implicit interactions that emerge and evolve over time. We need studies that investigate better ways to prevent major smart ecosystem failures due to implicit interactions among systems that behave according to their requirements and humans and social groups.
- *How do we model the behavior of human and social groups in the presence of evolving and adaptive behavior of smart AI components of human-centric ecosystems?* This requires new studies investigating how to capture behaviors that depend on human personality, gender, emotions, engagement, culture, ethic, age, and more. We need to develop ways to scale from models of humans to models of social groups to assess the quality of large smart, human-centric ecosystems.
- *How do we define self-adaptive strategies?* We need to carry out studies on how to better handle independently owned, developed, and maintained systems within a self-adaptive ecosystem. We also need to work our ways to harmonize ecosystem self-adaptive strategies with the evolving and adaptive behavior of evolving AI-powered systems within the ecosystem.
- *How do we define self-adaptive strategies for smart, human-centric ecosystems?* We need ways to better combine system and human models to include human and social behavior into ecosystem self-adaptation strategies. We need to carry out studies on leveraging AI to help to handle independently owned, developed, and maintained systems within a self-adaptive ecosystem. We also need to combine self-adaptive strategies with the evolving and non-deterministic behavior of AI-powered systems within the ecosystem. Ways to combine system and human models to include human and social behavior into ecosystem self-adaptation strategies need to be developed and trialed.
- *How do we predict and mitigate smart ecosystems failures that arise from implicit interactions of systems that behave correctly, according to their specifications?* We need to identify and harmonize corrective actions among independently owned and managed systems that behave according to sometimes contradicting requirements.

3.3 From NoOps to AIOps

3.3.1 Trends. A systematic study by Jabbari et al. [79] spotlights the key recent trends in DevOps: highly technical studies, like the study by Sven that focus on the qualities of Infrastructure-as-Code [81], organizational studies that reflects the collaborative dynamics behind DevOps [107], and studies on testing, like drill-like exercises contiguous to Chaos Engineering by Basiri et al. [16].

The focus on human-centric ecosystems introduces key challenges and complexity of the intrinsically socio-technical relation between the machines and the humans involved as well as the emergent ecology in between. Extreme organizational forms, such as NoOps by Gualtieri et al. [73], and operations managed by individual developers who carry out both Dev and Ops tasks together reduce the organizational space to the bare minimum.

NoOps refers to the objective of developers investing effort only in development activities, with operational activities limited to automatic synchronization, such as AIOps, and AI-driven operations [122]. At the time of writing, NoOps is still a vague concept with some operational



First number in the brackets: number of paragraphs in the analyzed literature that reflect the dimension.

Second number in the brackets: number of sources reflecting the dimension.

Fig. 7. A mind map diagram that highlights the main dimensions of the NoOps definition.

definitions that can be drawn from the literature [122], like the visual layout in Figure 7, tailored from [176].⁶

Figure 8, inspired from an exercise originally carried out by Correa,⁷ illustrates the consequences of highly individualistic organizational structures on the human-centric ecosystems, by comparing NoOps to classical DevOps structures. We can grasp two striking differences: (i) *automation* that inevitably reduces the role of human-driven operations and changes the relation to operations by developers, and (ii) *AI elements* that simplify the tasks of developers. Reducing human-driven operations forces the process to rely on automation for establishing and maintaining service continuity, while the complexity of human-centric ecosystems requires larger scale and hybrid computing infrastructures [113], with main challenges related to (i) the synergy that federated AI infrastructure governance mechanisms shall establish and maintain, and (ii) the interactions among humans and AI.

AIOps [122] leverages a blend of AI, ML, and data analytics to automate and enhance the software operational processes. AIOps aim to simplify and accelerate the collection, analysis, and management of data across the various IT strata to (i) improve operational efficiency, (ii) predict and prevent potential issues, and (iii) quickly deliver accurate services, with a complex intertwining of phases and steps. Figure 9—tailored from Jerry Lee⁸—well exemplifies the layers of complexity of even regular configurations of the AIOps phases and steps.

3.3.2 Open Challenges. Contextualizing the complexities of the regular configurations of the AIOps phases and steps in large human-centric ecosystems raises new questions: (a) How can we tackle the emerging synergies among organizations, humans, and software ecosystems? (b) How can AI strengthen and cater to those synergies? (c) How can we handle the impact of increasing dynamic machines and infrastructures, like the cloud continuum on operations in human-centric ecosystems? (d) How will humans relate to dynamically evolving ecosystems: Will a *PsyOps* discipline emerge in the software lifecycles to track the psychological well-being and fatigue-tracking of software developers and operators involved in complex and AI-synergistic software operations?

⁶ https://www.google.com/url?sa=t&source=web&rct=j&opi=89978449&url=https://research.tue.nl/files/199495365/Serban_D.pdf&ved=2ahUKEwi46rD11f2FAxWt_rsIHbArCgEQFnoECBEQAQ&usgAOvVaw3EV6SyNqJq4BEwxIRTL2Om

⁷ <https://medium.com/@vinuscorrea/noops-the-next-evolution-in-it-operations-or-just-another-stop-on-the-devops-journey-44dab6fb273d>

⁸ <https://tfir.io/aiops-a-long-way-to-get-there/>

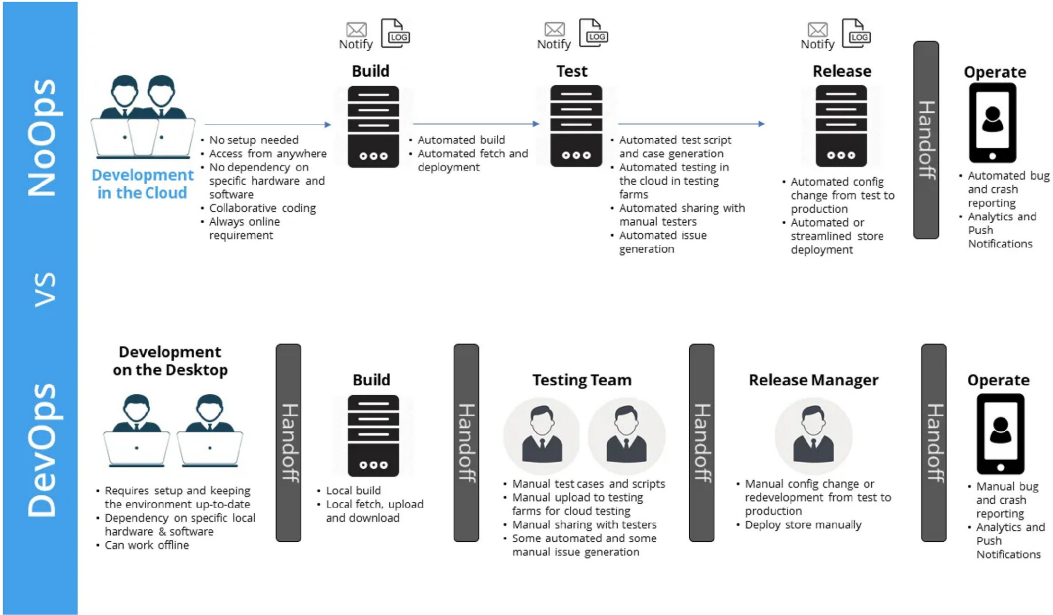
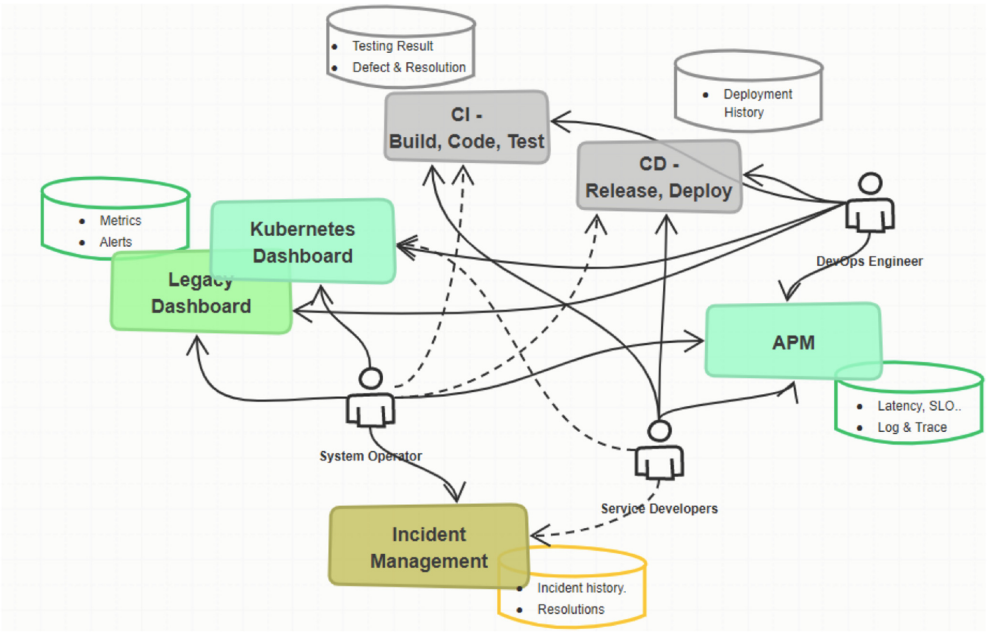


Fig. 8. A comparison of major DevOps and NoOps practices together; synergies and breaking-points are evident.



Straight arrows indicate data-flows.
Dotted arrows indicate responsibility allocation.
The Artificial Product Manager (APM) stands as the concentrator of infrastructure intelligence and automated operations management.

Fig. 9. AIOps in action, an overview of industrial solutions today.

The forthcoming opportunistic blend of AIOps configurations into human-centric ecosystems, in their highly individualistic forms, raises several challenges:

- *How can humans handle the increasing data complexity and volume?* We need new ways to manage and analyze vast amounts of data, as well as ensure the quality and consistency of data across multiple sources in complex IT environments.
- *How can we integrate humans into new AIOps systems?* We need to seamlessly integrate AIOps and humans with the software solutions and IT management tools and processes.
- *How do we maintain accurate AI models?* We need to develop and maintain accurate AI models that can adapt to changing IT environments. We need continuous learning and model tuning to keep up with new patterns and anomalies.
- *How can we enforce security and privacy by design?* We need to ensure data security and privacy compliance of AIOps platforms that access and process large volumes of sensitive data, while including humans in the loop.
- *How can we close the human-AI skill gap?* We need to reduce the large skill gap in the workforce when it comes to deploying and managing AIOps solutions as well as understanding and catering to the human-AI synergies [48].

Hybrid human ecosystems shall focus on how humans co-exist and co-create value across the multi-diversity of the cyberphysical infrastructures underlying complex ecosystems. For example, a smart city that is constituted by smart homes whose technical infrastructure runs on superconnected cyberphysical cloud continua [114] requires synergy between the various layers of the involved infrastructures as well as decentralized artificial and human intelligence operators. The maintenance and evolution of large organizational structure require new metrics to account for the emerging functional and non-functional properties, for instance, measuring the mutuality of information and knowledge-sharing among human and AI operators and measuring the mutual observability of operations for both human and AI operators.

The infrastructure marshaling and management responsibilities previously allocated to so-called orchestrators [81] shall now be delegated to constellations of orchestrators, with the increasing necessity of collaborating, harnessing the power of AI to interact, share knowledge, status updates, and operational procedures. In the scope of next-generation human-centric ecosystems, the classical dilemma of “orchestration vs. choreography”⁹ waves the so-called meta-orchestrators, that is, machines capable of harnessing multiple software-defined infrastructure languages, technologies, and hardware seamlessly while maintaining 100% operational control and observability.

A key question emerges in this complex scenario: “*What face would a meta-orchestrator of the future exhibit?*” Would established coding mechanisms such as reflexiveness [74] be part of the infrastructure programming models of the future? Would polyglot infrastructure programming encourage maintenance and evolution for infrastructures of the future? What are the coping mechanisms required for humans to blend their existence within the emergent scenarios of large ecosystems? Is there a risk of *AI fatigue* both for systems and for human strata? In summary, three grand challenges for the future arise:

PsyOps. The need for instrumented psychology software operations to fit humans into a complex human-centric ecosystems’ architecture. Hybrid AI-human interactions within the software lifecycle require new coordination models and psychological studies, beyond current metrics (like socio-technical congruence [108]) and *smells* (like community smells [128]). We need models and studies to understand both *human-AI congruence*, that is, the extent of continuity between human operations and AI assistance, and *AI fatigue*, that is, the excessive use of AI and

⁹<https://temporal.io/blog/to-choreograph-or-orchestrate-your-saga-that-is-the-question>

the consequences that this might have onto the stability of the organizational structure as well as the turmoil around the individual humans within it.

Meta-Orchestration. SE related to high-quality software systems' operations shall concentrate on two key ingredients: interchange/integration and AI-continuity. The challenge then becomes providing efficient, effective, and non-invasive engineering techniques to aid the construction and evaluation of meta-orchestrators-machines capable of blending multiple operations perspectives and languages together while maintaining operational awareness and capability over all such perspectives—augmented with AI which is rigged by—design to operate in full continuity with such meta-orchestrators, perhaps able to continuously learn from orchestration footprints striving to achieve provocative characteristics such as *explainable orchestration*.

Superconnectedness. From an operational perspective, software architectures and the underlying infrastructures are more and more reflecting software-defined infrastructure strata, which seem to strive toward fully connected *meshes* of both data and services to a point in which there is little to no distinction between data products and software services [133]. This characteristic reflects an extent of superconnectedness in the underlying infrastructure, namely the architecture opportunity that emerges by linking together directly all source and sink architecture elements thereby allowing an external-human or AI-operator to enter or govern the emerging ecosystem with novel interfaces beyond conventional CRUD-like operations. Emerging compute options such as *data meshes* [66] are already going in this direction. SE might play a more active role in the process of specifying and evaluating such systems in the future.

3.3.3 Roadmap. Although the main challenges that we discussed above seem rather self-contained, they are interrelated. For example, superconnectedness properties of a large-scale human-centric ecosystem reflect explicit and implicit requirements for meta-orchestration. While the challenges reflect complexities which we cannot fathom yet, SE principles and practices offer a richness of starting points. The following research questions emerge:

- *Can anthropometrics support psychological operations in SE?* Anthropometrics measures the human body and its cognitive instances and provides categorized data that can be used by (software) designers. We need to study the psychological strains in the software process, especially in relation to the continued use of AI and its interplay with software lifecycles.
- *How can AI fatigue be operationalized during AI-driven SE?* We need to study the consequences of the interplay that AI reflects upon the software lifecycle and developers' turnover, software process, product, and humans.
- *Are the principles of cloud and edge engineering valuable starting points to approach meta-orchestration research?* We need to define new principles of cloud and edge engineering to instrument future software designs and their operation in the context of an extreme interplay between humans (software-defined) infrastructures, AI, and cyberphysical systems strata.
- *Can explainable AI approaches be combined into explainable orchestration?* We need explainable AI approaches within explainable orchestration to expose both the traces of execution of software within the software lifecycle and the involvement of the AI components in the decisions.
- *How do we characterize human layers of meta-orchestration?* We need to define approaches to support human-centric governance schema behind integrated systems with human government.
- *How can we support the superconnectedness exerted in data meshes?* Data mesh engineering is itself an emerging discipline. We need to define rigorous and explainable approaches to instrument and maintain the superconnectedness, and promote data lineage and other data-* issues as center-stage software design issues.

- *How can we measure the cost of disconnectedness?* The negative instances of connectedness implies hidden costs well beyond our current understanding of technical debt. We need to define metrics of disconnectedness and approaches to models and management disconnectedness.

Each of the above open questions offers hints as to which related disciplines might be able to aid the scientific discourse by means of multi-disciplinarity. For example, cognitive psychology and/or cognitive ergonomics research would play very well within the scope of PsyOps research. Following these premises, at least two possible courses of action are conceivable. First, an endogenous perspective over research in the topics above may start looking for the aforementioned baselines, proceeding in finding a way to blend together such baselines synergistically, for example, blending together cloud continuum research with tiny-scale distributed or federated AIOps to achieve a higher understanding over superconnectedness. Second, an exogenous perspective over research in the topics above may look for opportunities in closely and no so closely related disciplines to understand whether any of the aforementioned challenges were ever encountered elsewhere.

4 2030 Research Horizon

Below we outline the 2030 research horizon as a combination of the main open issues that we have discussed in this article, and that comprise a general roadmap for engineering software systems in the era of emerging AI-powered systems and tools. Software is engineered by humans, for humans, and with humans. AI dramatically upsets the role of humans as software developers, as members of engineering teams, and as users, who become active elements of the human-centric software ecosystems and citizen software engineers. It challenges research and practice with new critical questions and important crosscutting concerns.

4.1 Key Research Questions

- (1) *What will software development and the SE profession look like in 2030 and beyond?*

AI-powered development of software systems spotlights a new research program of highly evolving development practices, tools, and profession. Core research direction include:

- (1.1) *Productivity.* How do we measure developers' and teams' productivity and experience in hybrid human-AI-powered-agent teams across the entire lifecycle? (see Section 2.1)
- (1.2) *Tools.* How do we better leverage software knowledge, domain knowledge, and development history to improve both engineering tools and AI for SE in general? (see Section 2.2)
- (1.3) *Profession.* How will we interact and work effectively with increasingly powerful AI-powered development agents for engineering useful software systems? (see Section 2.3)
- (1.4) *Education.* What will be the future software body of knowledge and curricula to attract and train excellent software engineers in 2030 and beyond? (see Section 2.5)

- (2) *What will future SE teams look like?*

AI-powered agents dramatically impact human-to-human, human-to-agent, and agent-to-agent communication, trust, and ethic, and redefine development teams:

- (2.1) *Human-human collaboration.* How will AI-powered agents impact human-human collaboration and communication? (see Section 2.3)
- (2.2) *Human-agent collaboration.* How will AI-powered agents impact human-agent and agent-agent collaboration and communication? (see Section 2.3)
- (2.3) *Developer-stakeholder communication.* How will AI-powered agents change developer-stakeholder and developer-developer empathy and barriers? (see Section 2.3)

- (2.4) *Creativity, trust, and distributed cognition.* What theories of human and AI-powered distributed cognition will we need to use to continue to boost creativity and trust? (see Section 2.3)
- (3) *What future SE processes will we see in 2030 and beyond?*
AI dramatically redefines the SE process:
 - (3.1) *Psychological operations in SE.* How will we support psychological operations and manage AI fatigue in SE? (see Section 3.3)
 - (3.2) *Orchestration.* How will we merge explainable AI approaches with cloud and edge principles into explainable meta-orchestration with human layers? (see Section 3.3)
 - (3.3) *Superconnectedness.* How will we measure and manage superconnectedness and disconnectedness in data meshes? (see Section 3.3)
- (4) *What will human-centric AI-powered software systems look like by 2030 and beyond?*
AI, XR, and the Internet of Things dramatically change the interactions between humans and AI-powered systems. They also upset assumptions about both systems that seamlessly adapt to each individual and systems-of-systems that must quickly evolve into human-centric smart ecosystems, where humans are an integral part and not mere users of the ecosystem:
 - (4.1) *Intelligent AUI.* What is the nature of future explainable AI-powered systems for effective and fair-by-design interactions with humans? (see Section 3.1)
 - (4.2) *Smart, human-centric ecosystems.* How will we engineer and verify future smart, human-centric ecosystem? (see Section 3.2)
 - (4.3) *Self-adaptive and evolving behavior.* How will we define and verify self-adaptive and evolving AI-powered ecosystems? (see Section 3.2)
 - (4.4) *Citizen software engineers.* How will we suitably educate citizen software engineers, and what are the kinds of AI-powered tools for citizen software engineers that will be needed? (see Section 2.4)

4.2 Crosscutting Concerns

Many emergent AI system challenges share some important crosscutting concerns about theory, transparency, ethics, and inclusiveness with an interdisciplinary approach:

- (1) *Theory.* What theories and metrics will we need to revisit? What can we borrow from other disciplines to capture the disruptive impact of AI on SE at all levels: development, productivity, experience, creativity, trust, autonomy, and evolution, with both humans and AI-powered agents in the process and the ecosystem?
- (2) *Transparency.* How will we suitably explain AI decisions for transparent interfaces, systems, processes, behavior, and evolution?
- (3) *Ethics.* Who is responsible for failures in future AI-heavy systems and processes? What are the key ethical SE issues introduced by AI-powered SE bots? How will we anticipate and prepare for bad actors as AI is adopted in software tools? How to protect human privacy and creativity?
- (4) *Inclusiveness.* How will we enforce fairness-by-design and respect human diversity and needs within AI-powered processes and products?

The new challenges of AI-powered engineering, AI-augmented socio-technical systems, and multifaceted ethical concerns such as fairness, human aspects, and technical dilemmas call for an escalation of inter-disciplinary collaborations with management sciences, HCI, behavioral psychology, social sciences social sciences, behavioral sciences, and cognitive psychology.

References

- [1] Ahmad Abdellatif, Khaled Badran, and Emad Shihab. 2020. MSRBot: Using bots to answer questions from software repositories. *Empirical Software Engineering* 25, 3 (2020), 1834–1863.
- [2] Silvia Abrahão, Emilio Insfran, Arthur Sluÿters, and Jean Vanderdonckt. 2021. Model-based intelligent user interface adaptation: Challenges and future directions. *Software and Systems Modeling* 20, 5 (2021), 1335–1349. DOI: <https://doi.org/10.1007/s10270-021-00909-7>
- [3] Bram Adams and Foutse Khomh. 2020. The diversity crisis of software engineering for artificial intelligence. *IEEE Software* 37, 5 (2020), 104–108.
- [4] Aakash Ahmad, Muhammad Waseem, Peng Liang, Mahdi Fahmideh, Mst Shamima Aktar, and Tommi Mikkonen. 2023. Towards human-bot collaborative software architecting with chatgpt. In *Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering*, 279–285.
- [5] Deniz Akdur. 2022. Analysis of software engineering skills gap in the industry. *ACM Transactions on Computing Education* 23, 1 (2022), 1–28.
- [6] Pierre A. Akiki. 2018. CHAIN: Developing model-driven contextual help for adaptive user interfaces. *Journal of Systems and Software* 135 (2018), 165–190. DOI: <https://doi.org/10.1016/J.JSS.2017.10.017>
- [7] Pierre A. Akiki, Arosha K. Bandara, and Yijun Yu. 2014. Adaptive model-driven user interface development systems. *ACM Computing Survey* 47, 1 (2014), Article 9, 1–33. DOI: <https://doi.org/10.1145/2597999>
- [8] Pierre A. Akiki, Arosha K. Bandara, and Yijun Yu. 2016. Engineering adaptive model-driven user interfaces. *IEEE Transactions on Software Engineering* 42, 12 (2016), 1118–1147. DOI: <https://doi.org/10.1109/TSE.2016.2553035>
- [9] Saleema Amershi, Daniel S. Weld, Mihaela Vorvoreanu, Adam Fournay, Besmira Nushi, Penny Collisson, Jina Suh, Shamsi T. Iqbal, Paul N. Bennett, Kori Inkpen, et al. 2019. Guidelines for human-AI interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. ACM, 1–13. DOI: <https://doi.org/10.1145/3290605.3300233>
- [10] Andrew Anderson, Jonathan Dodge, Amrita Sadarangani, Zoe Juozapaitis, Evan Newman, Jed Irvine, Souti Chattopadhyay, Matthew L. Olson, Alan Fern, and Margaret Burnett. 2020. Mental models of mere mortals with explanations of reinforcement learning. *ACM Transactions Interactive Intelligent Systems* 10, 2 (2020), Article 15, 1–37. DOI: <https://doi.org/10.1145/3366485>
- [11] Sumit Asthana, Hitesh Sajani, Elena Voyloshnikova, Birendra Acharya, and Kim Herzig. 2023. A case study of developer bots: Motivations, perceptions, and challenges. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 1268–1280.
- [12] Assaf Avishahar-Zeira and David H. Lorenz. 2023. Could no-code be code? Toward a no-code programming language for citizen developers. In *Proceedings of the 2023 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, 103–119.
- [13] Max Baarspul. 1990. A review of flight simulation techniques. *Progress in Aerospace Sciences* 27, 1 (1990), 1–120.
- [14] Gagan Bansal, Besmira Nushi, Ece Kamar, Daniel S. Weld, Walter S. Lasecki, and Eric Horvitz. 2019. Updates in human-AI teams: Understanding and addressing the performance/compatibility tradeoff. In *Proceedings of the 33th AAAI Conference on Artificial Intelligence*, 2429–2437.
- [15] Victor R. Basili and Barry T. Perricone. 1984. The influence of organizational factors on software quality and productivity. In *Proceedings of the 1st International Conference on Software Engineering*, 154–162.
- [16] Ali Basiri, Niosha Behnam, Ruud de Rooij, Lorin Hochstein, Luke Kosewski, Justin Reynolds, and Casey Rosenthal. 2016. Chaos engineering. *IEEE Software* 33, 3 (2016), 35–41. Retrieved from <http://dblp.uni-trier.de/db/journals/software/software33.html#BasiriBRHKRR16>
- [17] Kent Beck. 1999. *Extreme Programming Explained: Embrace Change*. Addison-Wesley Professional, Reading, MA.
- [18] Sarah Beecham, Helen Sharp, Tracy Hall, Nathan Baddoo, and Hugh Robinson. 2008. What do we know about developer motivation? *IEEE Software* 25, 4 (2008), 92–94. DOI: <https://doi.org/10.1109/MS.2008.105>
- [19] Antonia Bertolino, Pietro Braione, Guglielmo De Angelis, Luca Gazzola, Fitsum Meshesha Kifetew, Leonardo Mariani, Matteo Orrù, Mauro Pezzè, Roberto Pietrantuono, Stefano Russo, et al. 2022. A survey of field-based testing techniques. *ACM Computing Surveys* 54, 5 (2022), Article 92, 1–39. DOI: <https://doi.org/10.1145/3447240>
- [20] Björn Binzer and Till J. Winkler. 2022. Democratizing software development: A systematic multivocal literature review and research agenda on citizen development. In *Proceedings of the International Conference on Software Business*. Springer, 244–259.
- [21] Gordon S. Blair, Nelly Bencomo, and Robert B. France. 2009. Models@ run.time. *Computer* 42, 10 (2009), 22–27. DOI: <https://doi.org/10.1109/MC.2009.326>
- [22] Pierre Bourque, Richard E. Fairley, and IEEE Computer Society. 2014. *Guide to the Software Engineering Body of Knowledge (SWEBOK(R)): Version 3.0* (3rd ed.). IEEE Computer Society Press, Washington, DC, USA.
- [23] Frederick P. Brooks. 1995. *The Mythical Man-Month: Essays on Software Engineering* (anniversary ed.). Addison-Wesley Professional.

- [24] Margaret M. Burnett, Simone Stumpf, Jamie Macbeth, Stephann Makri, Laura Beckwith, Irwin Kwan, Anicia Peters, and Will Jernigan. 2016. GenderMag: A method for evaluating software's gender inclusiveness. *Interactive Computing* 28, 6 (2016), 760–787. DOI: <https://doi.org/10.1093/TWC/TWV046>
- [25] Jordi Cabot and Robert Clarisó. 2022. Low code for smart software development. *IEEE Software* 40, 1 (2022), 89–93.
- [26] Luiz Fernando Capretz, Pradeep Waychal, Jingdong Jia, Daniel Varona, and Yadira Lizama. 2021. International comparative studies on the software testing profession. *IT Professional* 23, 5 (2021), 56–61.
- [27] Edgar Ceh-Varela, Carlos Canto-Bonilla, and Dhimitraq Duni. 2023. Application of project-based learning to a software engineering course in a hybrid class environment. *Information and Software Technology* 158 (2023), 107189.
- [28] Alexander Chatzigeorgiou, Iftekar Ahmed, Haipeng Cai, Mauro Pezzè, and Denys Poshyvanyk. 2024. Artificial intelligence for software engineering: The journey so far and the road ahead. *ACM Transactions on Software Engineering and Methodology* 34, 9 (2024).
- [29] Rudrajit Choudhuri, Dylan Liu, Igor Steinmacher, Marco Gerosa, and Anita Sarma. 2024. How far are we? The triumphs and trials of generative AI in learning software engineering. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 1–13.
- [30] Orges Cico, Letizia Jaccheri, Anh Nguyen-Duc, and He Zhang. 2021. Exploring the intersection between software industry and software engineering education—A systematic mapping of software engineering trends. *Journal of Systems and Software* 172 (2021), 110736.
- [31] Paul Clements and Linda Northrop. 2002. *Software Product Lines*. Addison-Wesley Boston.
- [32] Hoa Khanh Dam, Truyen Tran, John Grundy, Aditya Ghose, and Yasutaka Kamei. 2019. Towards effective AI-powered agile project management. In *Proceedings of the 2019 IEEE/ACM 41st International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*. IEEE, 41–44.
- [33] Rogério de Lemos, Holger Giese, Hausi A. Müller, Mary Shaw, Jesper Andersson, Marin Litoiu, Bradley R. Schmerl, Gabriel Tamura, Norha M. Villegas, Thomas Vogel, et al. 2010. Software engineering for self-adaptive systems: A second research roadmap. In *Proceedings of the International Seminar on Software Engineering for Self-Adaptive Systems II*. Rogério de Lemos, Holger Giese, Hausi A. Müller, and Mary Shaw (Eds.), Lecture Notes in Computer Science, Vol. 7475, Springer, 1–32. DOI: https://doi.org/10.1007/978-3-642-35813-52_1
- [34] Ronnie de Souza Santos, William Das Neves Grillo, Djafran Cabral, Catarina De Castro, Nicole Albuquerque, and Cesar França. 2024. Post-pandemic hybrid work in software companies: Findings from an industrial case study. In *Proceedings of the 2024 IEEE/ACM 17th International Conference on Cooperative and Human Aspects of Software Engineering*, 68–78.
- [35] Ronnie E. de Souza Santos and Paul Ralph. 2022. A grounded theory of coordination in remote-first and hybrid software teams. In *Proceedings of the 44th International Conference on Software Engineering*, 25–35.
- [36] Tom DeMarco and Timothy Lister. 1999. *Peopleware: Productive Projects and Teams* (2nd ed.). Dorset House Publishing Co., New York.
- [37] Paul Denny, Viraj Kumar, and Nasser Giacaman. 2023. Conversing with copilot: Exploring prompt engineering for solving cs1 problems using natural language. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1*, 1136–1142.
- [38] Peter Totterdell/Dermot Browne and Mike Norman. 1990. *Adaptive User Interfaces: Principles*. Elsevier, USA.
- [39] Tilman Deuschel and Ted Scully. 2016. On the importance of spatial perception for the design of adaptive user interfaces. In *Proceedings of the 10th IEEE International Conference on Self-Adaptive and Self-Organizing Systems (SASO '16)*. IEEE Computer Society, 70–79. DOI: <https://doi.org/10.1109/SASO.2016.13>
- [40] Mateusz Dolata and Kevin Crowston. 2023. Making sense of AI systems development. *IEEE Transactions on Software Engineering* 50, 1 (2023), 123–140.
- [41] Schahram Dustdar, Stefan Nastic, and Ognjen Scekic. 2018. *Smart Cities: The Internet of Things, People and Systems* (1st ed.). Springer Publishing Company, Incorporated.
- [42] Noura El Moussa, Davide Molinelli, Mauro Pezzè, and Martin Tappler. 2021. Health of smart ecosystems. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '21)*. ACM, New York, NY, USA, 1491–1494. DOI: <https://doi.org/10.1145/3468264.3473137>
- [43] Omar Elazhary, Margaret-Anne Storey, Neil A. Ernst, and Elise Paradis. 2021. ADEPT: A socio-technical theory of continuous integration. In *Proceedings of the 43rd International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER '21)*. IEEE Press, 26–30. DOI: <https://doi.org/10.1109/ICSE-NIER52604.2021.00014>
- [44] Anthony Elliott, Brian Peiris, and Chris Parnin. 2015. Virtual reality in software engineering: Affordances, applications, and challenges. In *Proceedings of the 2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, Volume 2. IEEE, 547–550.

- [45] Linda Erlenhov, Francisco Gomes de Oliveira Neto, Riccardo Scandariato, and Philipp Leitner. 2019. Current and future bots in software development. In *Proceedings of the 2019 IEEE/ACM 1st International Workshop on Bots in Software Engineering (BotSE)*. IEEE, 7–11. DOI : <https://doi.org/10.1109/BotSE.2019.00009>
- [46] Farjam Eshraghian, Najmeh Hafezieh, Farveh Farivar, and Sergio de Cesare. 2024. AI in software programming: Understanding emotional responses to GitHub Copilot. *Information Technology and People* (2024). Retrieved from <https://doi-org.ezproxy.library.uvic.ca/10.1108/ITP-01-2023-0084>
- [47] Motahhare Eslami, Kristen Vaccaro, Min Kyung Lee, Amit Elazari Bar On, Eric Gilbert, and Karrie Karahalios. 2019. User attitudes towards algorithmic opacity and transparency in online reviewing platforms. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems (CHI '19)*. ACM, 494. DOI : <https://doi.org/10.1145/3290605.3300724>
- [48] Tobias Benjamin Fahse and Anuschka Schmitt. 2023. Exploring the synergies in human-AI hybrids: A longitudinal analysis in sales forecasting. In *Proceedings of the American Conference on Information Systems*. Association for Information Systems. Retrieved from <http://dblp.uni-trier.de/db/conf/amcis/amcis2023.html#FahseS23>
- [49] Robert Feldt, Francisco G. de Oliveira Neto, and Richard Torkar. 2018. Ways of applying artificial intelligence in software engineering. In *Proceedings of the 6th International Workshop on Realizing Artificial Intelligence Synergies in Software Engineering*, 35–41.
- [50] Nicole Forsgren, Jez Humble, and Gene Kim. 2018. *Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations*. IT Revolution Press, Portland, OR.
- [51] Nicole Forsgren, Margaret-Anne Storey, Chandra Maddila, Tom Zimmermann, Brian Houck, and Jenna Butler. 2021. The SPACE of developer productivity: There's more to it than you think. *Communications of the ACM* 64, 6 (2021), 20–48. DOI : <https://doi.org/10.1145/3454122.3454124>
- [52] Eduard Frankford, Clemens Sauerwein, Patrick Bassner, Stephan Krusche, and Ruth Breu. 2024. AI-tutoring in software engineering education. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering Education and Training*, 309–319.
- [53] Thordur Vikingur Fridgeirsson, Helgi Thor Ingason, Haukur Ingi Jonasson, and Hildur Jonsdottir. 2021. An authoritative study on the near future effect of artificial intelligence on project management knowledge areas. *Sustainability* 13, 4 (2021), 2345.
- [54] Ruti Gafni, Itzhak Aviv, Boris Kantsepolsky, Sofia Sherman, Havana Rika, Yariv Itzkovich, and Artem Barger. 2024. Objectivity by design: The impact of AI-driven approach on employees' soft skills evaluation. *Information and Software Technology* 170 (2024), 107430.
- [55] Krzysztof Z. Gajos and Krysta Chauncey. 2017. The influence of personality traits and cognitive load on the use of adaptive user interfaces. In *Proceedings of the 22nd International Conference on Intelligent User Interfaces (IUI '17)*. ACM, 301–306. DOI : <https://doi.org/10.1145/3025171.3025192>
- [56] Krzysztof Z. Gajos, Katherine Everitt, Desney S. Tan, Mary Czerwinski, and Daniel S. Weld. 2008. Predictability and accuracy in adaptive user interfaces. In *Proceedings of the 2008 Conference on Human Factors in Computing Systems (CHI)*. ACM, 1271–1274. DOI : <https://doi.org/10.1145/1357054.1357252>
- [57] Emily Arteaga Garcia, João Felipe Pimentel, Zixuan Feng, Marco Gerosa, Igor Steinmacher, and Anita Sarma. 2024. How to support ml end-user programmers through a conversational agent. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE)*. IEEE, 629–640.
- [58] David Garlan, Shang-Wen Cheng, and Bradley Schmerl. 2003. Increasing system dependability through architecture-based self-repair. In *Architecting Dependable Systems*. Rogério de Lemos, Cristina Gacek, and Alexander Romanovsky (Eds.), Springer Berlin Heidelberg, Berlin, 61–89.
- [59] Vahid Garousi, Gorkem Giray, and Eray Tuzun. 2019. Understanding the knowledge gaps of software engineers: An empirical analysis based on SWEBOK. *ACM Transactions on Computing Education* 20, 1 (2019), 1–33.
- [60] Daniel Gaspar-Figueiredo, Sílvia Abrahão, Marta Fernández-Diego, and Emilio Insfrán. 2023. A comparative study on reward models for UI adaptation with reinforcement learning. arXiv:2308.13937. Retrieved from <https://arxiv.org/abs/2308.13937>
- [61] Daniel Gaspar-Figueiredo, Sílvia Abrahão, Emilio Insfrán, and Jean Vanderdonckt. 2023. Measuring user experience of adaptive user interfaces using EEG: A replication study. In *Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering (EASE '23)*. ACM, 52–61. DOI : <https://doi.org/10.1145/3593434.3593452>
- [62] Daniel Gaspar-Figueiredo, Marta Fernández-Diego, Ruben Nuredini, Sílvia Abrahão, and Emilio Insfrán. 2024. Reinforcement learning-based framework for the intelligent adaptation of user interfaces. In *Proceedings of the 16th ACM SIGCHI Symposium on Engineering Interactive Computing Systems, EICS Companion 2024*. ACM, 40–48. DOI : <https://doi.org/10.1145/3660515.3661329>

- [63] Christoph Gebhardt, Brian Hecox, Bas van Opheusden, Daniel Wigdor, James Hillis, Otmar Hilliges, and Hrvoje Benko. 2019. Learning cooperative personalized policies from gaze data. In *Proceedings of the 32nd Annual ACM Symposium on User Interface Software and Technology (UIST '19)*. ACM, 197–208. DOI: <https://doi.org/10.1145/3332165.3347933>
- [64] Audrey Girouard, Erin Treacy Solovey, Leanne M. Hirshfield, Evan M. Peck, Krysta Chauncey, Angelo Sassaroli, Sergio Fantini, and Robert J. K. Jacob. 2010. From brain signals to adaptive interfaces: Using fNIRS in HCI. In *Brain-Computer Interfaces—Applying our Minds to Human-Computer Interaction*. Desney S. Tan and Anton Nijholt (Eds.), Springer, 221–237. DOI: https://doi.org/10.1007/978-1-84996-272-82_13
- [65] Camille Gobert, Kashyap Todi, Gilles Bailly, and Antti Oulasvirta. 2019. SAM: A modular framework for self-adapting web menus. In *Proceedings of the 24th International Conference on Intelligent User Interfaces (IUI '19)*. ACM, 481–484. DOI: <https://doi.org/10.1145/3301275.3302314>
- [66] Abel Goedegebuure, Indika Kumara, Stefan Driessen, Dario Di Nucci, Geert Monsieur, Willem Jan van den Heuvel, and Damian Andrew Tamburri. 2024. Data mesh: A systematic gray literature review. arXiv:2304.01062. Retrieved from <https://arxiv.org/abs/2304.01062>
- [67] Valentina Golendukhina, Valentina Lenarduzzi, and Michael Felderer. 2022. What is software quality for AI engineers? Towards a thinning of the fog. In *Proceedings of the 1st International Conference on AI Engineering: Software Engineering for AI*, 1–9.
- [68] Don Gotterbarn, Keith Miller, and Simon Rogerson. 1997. Software engineering code of ethics. *Communications of the ACM* 40, 11 (1997), 110–118.
- [69] Daniel Grazier, Fabian Fagerholm, Xiaofeng Wang, and Pekka Abrahamsson. 2018. What happens when software developers are (un)happy. *Journal of Systems and Software* 140 (2018), 32–47. DOI: <https://doi.org/10.1016/j.jss.2018.02.041>
- [70] Shalom Greene, Himanshu Thapliyal, and Allison Caban-Holt. 2016. A survey of affective computing for stress detection: Evaluating technologies in stress detection for better health. *IEEE Consumer Electronics Magazine* 5, 4 (2016), 44–56. DOI: <https://doi.org/10.1109/MCE.2016.2590178>
- [71] Michaela Greiler, Margaret-Anne Storey, and Abi Noda. 2023. An actionable framework for understanding and improving developer experience. *IEEE Transactions on Software Engineering* 49, 4 (2023), 1411–1425. DOI: <https://doi.org/10.1109/TSE.2023.1234567>
- [72] John C. Grundy. 2020. Human-centric software engineering for next generation cloud- and edge-based smart living applications. In *Proceedings of the 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGRID '20)*. IEEE, 1–10. DOI: <https://doi.org/10.1109/CCGRID49817.2020.00-93>
- [73] Mike Gualtieri, Sandy Carielli, and Cheryl McKinnon. 2011. I Don't Want DevOps. I Want NoOps. Retrieved from https://go.forrester.com/blogs/11-02-07-i_dont_want_devops_i_want_noops/
- [74] Kris Gybels, Roel Wuyts, Stéphane Ducasse, and Maja D'Hondt. 2006. Inter-language reflection: A conceptual model and its implementation. *Computer Languages, Systems and Structures* 32, 2-3 (2006), 109–124. DOI: <https://doi.org/10.1016/j.cl.2005.10.003>
- [75] Khadija Hanifi, Orcun Cetin, and Cemal Yilmaz. 2023. On ChatGPT: Perspectives from software engineering students. In *Proceedings of the 2023 IEEE 23rd International Conference on Software Quality, Reliability, and Security (QRS)*. IEEE, 196–205.
- [76] Catherine Hicks, Carol Lee, and Kristen Foster-Marks. 2024. The new developer: AI skill threat, identity change; developer thriving in the transition to AI-assisted software development. DOI: <https://doi.org/10.31234/osf.io/2gej5>
- [77] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large language models for software engineering: A systematic literature review. arXiv:2308.10620. Retrieved from <https://arxiv.org/abs/2308.10620>
- [78] Andrew Hunt and David Thomas. 1999. *The Pragmatic Programmer: From Journeyman to Master*. Addison-Wesley Professional, Boston, MA.
- [79] Ramtin Jabbari, Nauman Bin Ali, Kai Petersen, and Binish Tanveer. 2016. What is DevOps? A systematic mapping study on definitions and practices. In *Proceedings of the Scientific Workshop Proceedings of XP2016 (Edinburgh, Scotland, UK) (XP '16 Workshops)*. ACM, New York, NY, USA, Article 12, 1–11. DOI: <https://doi.org/10.1145/2962695.2962707>
- [80] Victoria Jackson, Bogdan Vasilescu, Daniel Russo, Paul Ralph, Rafael Prikladnicki, Maliheh Izadi, Sarah D'Angelo, Sarah Inman, Anielle Lisboa, and André van der Hoek. 2025. The impact of generative AI on creativity in software development: A research agenda. *ACM Transaction on Software Engineering and Methodology* 34, 4 (April 2025).
- [81] SvenJohann. 2017. Kief Morris on infrastructure as code. *IEEE Software* 34, 1 (2017), 117–120. Retrieved from <http://dblp.uni-trier.de/db/journals/software/software34.html#Johann17>
- [82] Capers Jones. 1986. Measuring programmer productivity. *IBM Systems Journal* 22, 1 (1986), 36–48.
- [83] Eirini Kalliamvakou, Christian Bird, Thomas Zimmermann, Andrew Begel, Robert DeLine, and Daniel M German. 2017. What makes a great manager of software engineers? *IEEE Transactions on Software Engineering* 45, 1 (2017), 87–106.

- [84] Eirini Kalliamvakou, Sida Peng, Peter Cihon, and Mert Demirer. 2023. The impact of AI on developer productivity: Evidence from GitHub Copilot. *Communications of the ACM* 66, 6 (2023), 28–37. DOI : <https://doi.org/10.1145/3454122.3454124>
- [85] Christian Kästner and Eunsuk Kang. 2020. Teaching software engineering for AI-enabled systems. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET '20)*. ACM, New York, NY, USA, 45–48. DOI : <https://doi.org/10.1145/3377814.3381714>
- [86] Brian W. Kernighan and Dennis M. Ritchie. 1988. *The C Programming Language* (2nd ed.). Prentice Hall, Englewood Cliffs, NJ.
- [87] Dron Khanna, Emily Laue Christensen, Saagarika Gosu, Xiaofeng Wang, and Maria Paasivaara. 2024. Hybrid work meets agile software development: A systematic mapping study. In *Proceedings of the 2024 IEEE/ACM 17th International Conference on Cooperative and Human Aspects of Software Engineering*, 57–67.
- [88] Vassilka D. Kirova, Cyril S. Ku, Joseph R. Laracy, and Thomas J. Marlowe. 2024. Software engineering education must adapt and evolve for an LLM environment. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1*, 666–672.
- [89] Antti Knutas, Victoria Palacin, Giovanni Maccani, and Markus Helfert. 2019. Software engineering in civic tech a case study about code for Ireland. In *Proceedings of the 2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Society (ICSE-SEIS)*. IEEE, 41–50.
- [90] Kathrin Komp-Leukkunen. 2024. How ChatGPT shapes the future labour market situation of software engineers: A Finnish Delphi study. *Futures* 160 (2024), 103382.
- [91] Mohammad Amin Kuhail, Sujith Samuel Mathew, Ashraf Khalil, Jose Berengueres, and Syed Jawad Hussain Shah. 2024. “Will I be replaced?” Assessing ChatGPT’s effect on software development and programmer perceptions of AI tools. *Science of Computer Programming* 235 (2024), 103111.
- [92] Todd Kulesza, Margaret M. Burnett, Weng-Keen Wong, and Simone Stumpf. 2015. Principles of explanatory debugging to personalize interactive machine learning. In *Proceedings of the 20th International Conference on Intelligent User Interfaces (IUI '15)*. ACM, 126–137. DOI : <https://doi.org/10.1145/2678025.2701399>
- [93] Kati Kuusinen, Helen Petrie, Fabian Fagerholm, and Tommi Mikkonen. 2016. Flow, intrinsic motivation, and developer experience in software engineering. In *Agile Processes, in Software Engineering, and Extreme Programming*. Helen Sharp and Tracy Hall (Eds.). Springer International Publishing, Cham, 104–117.
- [94] Samuli Laato, Matti Mäntymäki, A. K. M. Najmul Islam, Sami Hyrynsalmi, and Teemu Birkstedt. 2023. Trends and trajectories in the software industry: Implications for the future of work. *Information Systems Frontiers* 25, 2 (2023), 929–944.
- [95] Samuli Laato, Miika Tiainen, A. K. M. Najmul Islam, and Matti Mäntymäki. 2022. How to explain AI systems to end users: A systematic literature review and research agenda. *Internet Research* 32, 7 (2022), 1–31.
- [96] An Ngoc Lam, Anh Tuan Nguyen, Hoan Anh Nguyen, and Tien N Nguyen. 2017. Bug localization with combination of deep learning and information retrieval. In *Proceedings of the 2017 IEEE/ACM 25th International Conference on Program Comprehension (ICPC)*. IEEE, 218–229.
- [97] Maxime Lamothe, Yann-Gaël Guéhéneuc, and Weiyi Shang. 2021. A systematic review of API evolution literature. *ACM Computing Surveys* 54, 8 (2021), 1–36.
- [98] Thomas Langerak, Sammy Joe Christen, Mert Albaba, Christoph Gebhardt, and Otmar Hilliges. 2022. MARLUI: Multi-agent reinforcement learning for goal-agnostic adaptive UIs. arXiv:2209.12660. Retrieved from <https://arxiv.org/abs/2209.12660>
- [99] Filippo Lanubile, Christof Ebert, Rafael Prikladnicki, and Aurora Vizcaino. 2010. Collaboration tools for global software engineering. *IEEE Software* 27, 2 (2010), 52.
- [100] Talia Lavie and Joachim Meyer. 2010. Benefits and costs of adaptive user interfaces. *International Journal of Human-Computer Studies* 68, 8 (2010), 508–524. DOI : <https://doi.org/10.1016/j.ijhcs.2010.01.004>
- [101] Paul Luo Li, Amy J. Ko, and Jiamin Zhu. 2015. What makes a great software engineer? In *Proceedings of the 2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*. IEEE, 700–710.
- [102] Tianyi Li, Mihaela Vorvoreanu, Derek DeBellis, and Saleema Amershi. 2023. Assessing human-AI interaction early through factorial surveys: A study on the guidelines for human-AI interaction. *ACM Transactions Computer Human Interactions* 30, 5 (2023), Article 69, 1–45. DOI : <https://doi.org/10.1145/3511605>
- [103] David Lo. 2023. Trustworthy and synergistic artificial intelligence for software engineering: Vision and roadmaps. In *Proceedings of the International Conference on Software Engineering: Future of Software Engineering*. IEEE, 69–85.
- [104] Qinghua Lu, Liming Zhu, Xiwei Xu, Jon Whittle, David Douglas, and Conrad Sanderson. 2022. Software engineering for responsible AI: An empirical study and operationalised patterns. In *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice*, 241–242.

- [105] Qinghua Lu, Liming Zhu, Xiwei Xu, Jon Whittle, Didar Zowghi, and Aurelie Jacquet. 2023. Responsible AI pattern catalogue: A collection of best practices for ai governance and engineering. *ACM Computing Surveys* 56, 7 (2023), Article 173, 1–35.
- [106] Silverio Martínez-Fernández, Justus Bogner, Xavier Franch, Marc Oriol, Julien Siebert, Adam Trendowicz, Anna Maria Vollmer, and Stefan Wagner. 2022. Software engineering for AI-based systems: a survey. *ACM Transactions on Software Engineering and Methodology* 31, 2 (2022), 1–59.
- [107] T. Masombuka and Ernest Mnkandla. 2018. A DevOps collaboration culture acceptance model. In *Proceedings of the Annual Conference of the South African Institute of Computer Scientists and Information Technologists*. ACM, 279–285. Retrieved from <http://dblp.uni-trier.de/db/conf/saicsit/saicsit2018.html#MasombukaM18>
- [108] Wolfgang Mauere, Mitchell Joblin, Damian A. Tamburri, Carlos Paradis, Rick Kazman, and Sven Apel. 2022. In search of socio-technical congruence: A large-scale longitudinal study. *IEEE Transactions on Software Engineering* 48, 8 (2022), 3159–3184. DOI : <https://doi.org/10.1109/TSE.2021.3082074>
- [109] Marshall McLuhan. 1977. Laws of the Media. *ETC: A Review of General Semantics* 34, 2 (1977), 173–179. Retrieved from <http://www.jstor.org/stable/42575246>
- [110] Leonel Merino, Mircea Lungu, and Christoph Seidl. 2020. Unleashing the potentials of immersive augmented reality for software engineering. In *Proceedings of the 2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 517–521.
- [111] Marjan Mernik, Jan Heering, and Anthony M. Sloane. 2005. When and how to develop domain-specific languages. *ACM Computing Surveys* 37, 4 (2005), 316–344.
- [112] Nesrine Mezhoudi and Jean Vanderdonckt. 2021. Toward a task-driven Intelligent GUI adaptation by mixed-initiative. *International Journal on Human Computer Interactions* 37, 5 (2021), 445–458. DOI : <https://doi.org/10.1080/10447318.2020.1824742>
- [113] Aleksandar Milenkoski, Alexandru Iosup, Samuel Kounev, Kai Sachs, Piotr Rygielski, Jason Ding, Walfredo Cirne, and Florian Rosenberg. 2013. Cloud Usage Patterns: A Formalism for Description of Cloud Usage Scenarios. Technical Report SPEC-RG-2013-001 v.1.0.1. SPEC Research Group—Cloud Working Group, Standard Performance Evaluation Corporation (SPEC), Gainesville, VA, USA. Retrieved from http://research.spec.org/fileadmin/user_upload/documents/rg_cloud/endorsed_publications/SPEC-RG-2013-001_CloudUsagePatterns.pdf
- [114] Dejan S. Milojicic. 2020. The edge-to-cloud continuum. *IEEE Computer* 53, 11 (2020), 16–25. Retrieved from <http://dblp.uni-trier.de/db/journals/computer/computer53.html#Milojicic20a>
- [115] Ivan Mistrík, John Grundy, Andre Van der Hoek, and Jim Whitehead. 2010. *Collaborative Software Engineering: Challenges and Prospects*. Springer.
- [116] Hausi A. Müller, Mauro Pezzè, and Mary Shaw. 2008. Visibility of control in adaptive systems. In *Proceedings of the 2nd International Workshop on Ultra-Large-Scale Software-Intensive Systems (ULSSIS@ICSE '08)*. ACM, 23–26. DOI : <https://doi.org/10.1145/1370700.1370707>
- [117] Alex Murphy, Ben Kelly, Kai Bergmann, Kyrylo Khaletskyy, Rory V. O'Connor, and Paul M. Clarke. 2019. Examining unequal gender distribution in software engineering. In *Proceedings of the 26th European Conference on Systems, Software and Services Process Improvement (EuroSPI '19)*. Springer, 659–671.
- [118] Peter Naur. 1986. Programming as theory building. *Microprocessing and Microprogramming* 15, 5 (1986), 253–261.
- [119] Peter Naur and Brian Randell. 1969. *Software Engineering: Report on a Conference Sponsored by the NATO Science Committees*. Technical Report. Scientific Affairs Division, NATO.
- [120] A.F. Norcio and J. Stanley. 1989. Adaptive human-computer interfaces: A literature survey and perspective. *IEEE Transactions on Systems, Man, and Cybernetics* 19, 2 (1989), 399–408. DOI : <https://doi.org/10.1109/21.31042>
- [121] L. Northrop, Peter Feiler, R. P. Gabriel, John Goodenough, Rick Linger, Thomas Longstaff, Rick Kazman, M. Klein, Douglas Schmidt, Kevin Sullivan, et al. 2006. *Ultra-Large-Scale Systems—The Software Challenge of the Future*. Technical Report. Software Engineering Institute—Carnegie Mellon.
- [122] Paolo Notaro, Jorge Cardoso, and Michael Gerndt. 2020. A systematic mapping study in AIOps. In *Proceedings of the ICSSOC Workshops*. Hakim Hacid, Fatma Outay, Hye Young Paik, Amira Alloum, Marinella Petrocchi, Mohamed Reda Bouadjene, Amin Beheshti, Xumin Liu, and Abderrahmane Maaradji (Eds.), Lecture Notes in Computer Science, Vol. 12632, Springer, 110–123. Retrieved from <http://dblp.uni-trier.de/db/conf/icsoc/icsoc2020w.html#NotaroCG20>
- [123] Nargess Nourbakhsh, Fang Chen, Yang Wang, and Rafael A. Calvo. 2017. Detecting users' cognitive load by galvanic skin response with affective interference. *ACM Transactions Interactions on Intelligent Systems* 7, 3 (2017), Article 12, 1–20. DOI : <https://doi.org/10.1145/2960413>
- [124] John P. Oliver and Srivastava Sanjay. 1999. The Big Five Trait taxonomy: History, measurement, and theoretical perspectives. In *Handbook of Personality: Theory and Research*. L. A. Pervin and O. P. John (Eds.). Guilford Press, 102–138.
- [125] John P. Oliver and Srivastava Sanjay. 2008. *Perspectives on Personality* (6th ed.). Pearson.

- [126] Marten Oltrogge, Erik Derr, Christian Stransky, Yasemin Acar, Sascha Fahl, Christian Rossow, Giancarlo Pellegrino, Sven Bugiel, and Michael Backes. 2018. The rise of the citizen developer: Assessing the security impact of online app generators. In *Proceedings of the 2018 IEEE Symposium on Security and Privacy (SP)*. IEEE, 634–647.
- [127] Ipek Ozkaya. 2022. A paradigm shift in automating software engineering tasks: Bots. *IEEE Software* 39, 5 (2022), 4–8.
- [128] Fabio Palomba, Damian Andrew Tamburri, Francesca Arcelli Fontana, Rocco Oliveto, Andy Zaidman, and Alexander Serebrenik. 2021. Beyond technical aspects: How do community smells influence the intensity of code smells? *IEEE Transactions on Software Engineering* 47, 1 (2021), 108–129. DOI: <https://doi.org/10.1109/TSE.2018.2883603>
- [129] Aastha Pant, Rashina Hoda, Chakkrit Tantithamthavorn, and Burak Turhan. 2024. Ethics in AI through the practitioner’s view: A grounded theory literature review. *Empirical Software Engineering* 29, 3 (2024), 67.
- [130] Chris Paton, Andre W. Kushniruk, Elizabeth M. Borycki, Mike English, and Jim Warren. 2021. Improving the usability and safety of digital health systems: The role of predictive human-computer interaction modeling. *Journal of Medical Internet Research* 23, 5 (May 2021), e25281.
- [131] Sampo V. Paunonen. 2003. Big Five factors of personality and replicated predictions of behavior. *Journal of Personality and Social Psychology* 84, 2 (2003), 411–424. DOI: <https://doi.org/10.1037/0022-3514.84.2.411>
- [132] Olga Petrovska, Lee Clift, Faron Moller, and Rebecca Pearsall. 2024. Incorporating generative AI into software development education. In *Proceedings of the 8th Conference on Computing Education Practice*, 37–40.
- [133] Giovanni Quattrocchi, Willem-Jan van den Heuvel, and Damian Andrew Tamburri. 2024. The data product-service composition frontier: A hybrid learning approach. *ACM Transactions on Management Information Systems* 15, 1 (March 2024), Article 6, 22 pages. DOI: <https://doi.org/10.1145/3649319>
- [134] Emilee J. Rader, Kelley Cotter, and Janghee Cho. 2018. Explanations as mechanisms for supporting algorithmic transparency. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems (CHI '18)*. ACM, 103. DOI: <https://doi.org/10.1145/3173574.3173677>
- [135] Charles Rich and Richard Waters. 1988. The programmer’s apprentice project: A research overview. *Computer* 21, 11 (1988), 10–25. DOI: <https://doi.org/10.1109/2.86782>
- [136] Tim Rietz. 2019. Designing a conversational requirements elicitation system for end-users. In *Proceedings of the 2019 IEEE 27th International Requirements Engineering Conference (RE)*. IEEE, 452–457.
- [137] Pierre N. Robillard and Martin P. Robillard. 2000. Types of collaborative work in software engineering. *Journal of Systems and Software* 53, 3 (2000), 219–224.
- [138] Gema Rodríguez-Pérez, Reza Nadri, and Meiyappan Nagappan. 2021. Perceived diversity in software engineering: A systematic literature review. *Empirical Software Engineering* 26 (2021), 1–38.
- [139] Babak Darvish Rouhani, Mohd Naz’ri Mahrin, Fatemeh Nikpay, Rodina Binti Ahmad, and Pourya Nikfard. 2015. A systematic literature review on enterprise architecture implementation methodologies. *Information and Software Technology* 62 (2015), 1–20.
- [140] Daniel Russo. 2024. Navigating the complexity of generative AI adoption in software engineering. *ACM Transactions on Software Engineering and Methodology* 33, 5 (2024), Article 135, 1–50.
- [141] Elizabeth B.-N. Sanders and Pieter Jan Stappers. 2008. Co-creation and the new landscapes of design. *CoDesign* 4, 1 (2008), 5–18. DOI: <https://doi.org/10.1080/15710880701875068>
- [142] Jaakko Sauvola, Sasu Tarkoma, Mika Klemettinen, Jukka Riekk, and David Doermann. 2024. Future of software development with generative AI. *Automated Software Engineering* 31, 1 (2024), 26.
- [143] J. Schneider-Hufschmidt, T. Kühme, and MalinowskiU. 1993. *Adaptive User Interfaces: Principles and Practice*. North Holland, London.
- [144] Concetta Semeraro, Mario Lezoche, Hervé Panetto, and Michele Dassisti. 2021. Digital twin paradigm: A systematic literature review. *Computers in Industry* 130 (2021), 103469.
- [145] Rifat Ara Shams, Didar Zowghi, and Muneera Bano. 2023. AI and the quest for diversity and inclusion: A systematic literature review. *AI and Ethics* 5 (2023), 1–28.
- [146] Ben Shneiderman. 2020. Bridging the gap between ethics and practice: Guidelines for reliable, safe, and trustworthy human-centered AI systems. *ACM Transactions on Interactive Intelligent Systems* 10, 4 (2020), 1–31.
- [147] James Shore and Shane Warden. 2007. *The Art of Agile Development*. O’Reilly Media, Sebastopol, CA.
- [148] Martin Stancek, Ivan Polasek, Tibor Zalabai, Juraj Vincur, Rodi Jolak, and Michel Chaudron. 2024. Collaborative software design and modeling in virtual reality. *Information and Software Technology* 166 (2024), 107369.
- [149] Vladimir Stantchev, Olaf Radant, and Ricardo Colomo-Palacios. 2016. Assessment of continuing educational measures in software engineering: A view from the industry. *International Journal of Engineering Education* 32, 2B (2016), 905–914.
- [150] Åsne Stige, Efraxia D. Zamani, Patrick Mikalef, and Yuzhen Zhu. 2023. Artificial intelligence (AI) for user experience (UX) design: A systematic literature review and future research agenda. *Information Technology and People* 37, 6 (2023), 2324–2352.

- [151] Margaret-Anne Storey, Daniel Russo, Nicole Novielli, Takashi Kobayashi, and Dong Wang. 2024. A disruptive research playbook for studying disruptive innovations. *ACM Transactions on Software Engineering and Methodology* 33, 8, Article 195 (2024), 1–29.
- [152] Margaret-Anne Storey, Christoph Treude, Arie Van Deursen, and Li-Te Cheng. 2010. The impact of social media on software engineering practices and tools. In *Proceedings of the FSE/SDP Workshop on Future of Software Engineering Research*, 359–364.
- [153] Margaret-Anne Storey, Thomas Zimmermann, Christian Bird, Jacek Czerwinka, Brendan Murphy, and Eirini Kalliamvakou. 2021. Towards a theory of software developer job satisfaction and perceived productivity. *IEEE Transactions on Software Engineering* 47, 10 (2021), 2125–2142. DOI: <https://doi.org/10.1109/TSE.2019.2944354>
- [154] R. Subramanyam and M. S. Krishnan. 2003. Understanding and improving software productivity: An empirical study. *Communications of the ACM* 46, 2 (2003), 73–77.
- [155] Thomas Süße, Maria Kobert, Simon Grapenthin, and Bernd-Friedrich Voigt. 2023. AI-powered chatbots and the transformation of work: Findings from a case study in software development and software engineering. In *Proceedings of the Working Conference on Virtual Enterprises*. Springer, 689–705.
- [156] Stephanie D. Teasley, Lisa A. Covi, Mayuram S. Krishnan, and Judith S. Olson. 2002. Rapid software development through team collocation. *IEEE Transactions on Software Engineering* 28, 7 (2002), 671–683.
- [157] Kashyap Todi, Gilles Bailly, Luis Leiva, and Antti Oulasvirta. 2021. Adapting user interfaces with model-based reinforcement learning. In *Proceedings of the 2021 Conference on Human Factors in Computing Systems (CHI '21)*. ACM, New York, NY, USA, Article 573, 13 pages. DOI: <https://doi.org/10.1145/3411764.3445497>
- [158] Kashyap Todi and Tanya R. Jonker. 2023. A framework for computational design and adaptation of extended reality user interfaces. arXiv:2309.04025. Retrieved from <https://arxiv.org/abs/2309.04025>
- [159] James E. Trumbly, Kirk P. Arnett, and Peter C. Johnson. 1994. Productivity gains via an adaptive user interface: An empirical analysis. *International Journal of Human-Computer Studies* 40, 1 (1994), 63–81. DOI: <https://doi.org/10.1006/ijhc.1994.1004>
- [160] Michele Tufano, Anisha Agarwal, Jinu Jang, Roshanak Zilouchian Moghaddam, and Neel Sundaresan. 2024. AutoDev: Automated AI-driven development. arXiv:2403.08299. Retrieved from <https://arxiv.org/abs/2403.08299>
- [161] Marcel Valový and Alena Buchalceva. 2023. The psychological effects of AI-assisted programming on students and professionals. In *Proceedings of the 2023 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 385–390.
- [162] Jean Vanderdonckt, Sara Bouzit, Gaëlle Calvary, and Denis Chêne. 2020. Exploring a design space of graphical adaptive menus: Normal vs. small screens. *ACM Transactions Interactive Intelligent Systems* 10, 1 (2020), Article 2, 1–40. DOI: <https://doi.org/10.1145/3237190>
- [163] Stefan Wagner and Melanie Ruhe. 2019. A systematic review of productivity factors in software development. arXiv:1801.06475. Retrieved from <https://arxiv.org/abs/1801.06475>
- [164] Zhiyuan Wan, Xin Xia, David Lo, and Gail C. Murphy. 2019. How does machine learning change software development practices? *IEEE Transactions on Software Engineering* 47, 9 (2019), 1857–1871.
- [165] Junjie Wang, Ye Yang, Song Wang, Jun Hu, and Qing Wang. 2022. Context-and fairness-aware in-process crowdworker recommendation. *ACM Transactions on Software Engineering and Methodology* 31, 3 (2022), 1–31.
- [166] Wei Wang, Hourieh Khalajzadeh, John C. Grundy, Anuradha Madugalla, and Humphrey O. Obie. 2024. Adaptive user interfaces for software supporting chronic disease. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Society (ICSE-SEIS '24)*. ACM, 118–129. DOI: <https://doi.org/10.1145/3639475.3640104>
- [167] Yi Wang and David Redmiles. 2019. Implicit gender biases in professional software development: An empirical study. In *Proceedings of the 2019 IEEE/ACM 41st International Conference on Software Engineering: Software engineering in Society (ICSE-SEIS)*. IEEE, 1–10.
- [168] Barbara Weber, Thomas Fischer, and René Riedl. 2021. Brain and autonomic nervous system activity measurement in software engineering: A systematic literature review. *Journal of Systems and Software* 178 (2021), 110946. DOI: <https://doi.org/10.1016/j.jss.2021.110946>
- [169] Gerald M. Weinberg. 1971. *The Psychology of Computer Programming*. Van Nostrand Reinhold, New York.
- [170] Justin D. Weisz, Michael Muller, Stephanie Houde, John Richards, Steven I. Ross, Fernando Martinez, Mayank Agarwal, and Kartik Talamadupula. 2021. Perfection not required? Human-AI partnerships in code translation. In *Proceedings of the 26th International Conference on Intelligent User Interfaces*, 402–412.
- [171] Jim Whitehead. 2007. Collaboration in software engineering: A roadmap. In *Proceedings of the Future of Software Engineering (FOSE '07)*. IEEE, 214–225.
- [172] Enes Yigitbas, Ivan Jovanovikj, Kai Biermeier, Stefan Sauer, and Gregor Engels. 2020. Integrated model-driven development of self-adaptive user interfaces. *Software and Systems Modeling* 19, 5 (2020), 1057–1081. DOI: <https://doi.org/10.1007/S10270-020-00777-7>

- [173] Enes Yigitbas, Hagen Stahl, Stefan Sauer, and Gregor Engels. 2017. Self-adaptive UIs: Integrated model-driven development of UIs and their adaptations. In *Proceedings of the 13th European Conference on Modelling Foundations and Applications (ECMFA@STAF '17)*. Anthony Anjorin and Huáscar Espinoza (Eds.), Lecture Notes in Computer Science, Vol. 10376, Springer, 126–141. DOI: https://doi.org/10.1007/978-3-319-61482-3_8
- [174] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. AutoCodeRover: Autonomous program improvement. arXiv:2404.05427. Retrieved from <https://arxiv.org/abs/2404.05427>
- [175] Lamia Zouhaier, Yosra Ben Dali Hlaoui, and Leila Ben Ayed. 2023. Adaptive user interface based on accessibility context. *Multimedia Tools and Applications* 82, 23 (2023), 35621–35650. DOI: <https://doi.org/10.1007/S11042-023-14390-5>
- [176] Dragos Mihai Șerban. 2021. *Demystifying NoOps: Operational Model, Challenges and Insights from the Trenches*. Master's Thesis. Eindhoven University of Technology.

Received 14 September 2024; revised 14 September 2024; accepted 18 November 2024